OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TECHNOLOGICAL UNIVERSITY DUBLIN

Open Source Firewalls Assignment

- Danyil Tymchuk (B00167321)
- Illia Stefanovskyi (B00165280)
- Artem Surzhenko (B00163362)

Table of Contents

Table of Contents.....1
Submission Coversheet.....2
Declaration.....3
Scenario and Network Topology.....4
Test Plan.....6
Nftables.....7
pfSense.....19
OPNsense.....31
Firewall Comparison.....42
References.....43
Appendix: Comprehensive Test Plan.....44
Appendix: Commands and Configurations (nftables).....52

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Assessment Information

Assessment:	CA1 – Open Source Firewalls Assignment		
Module:	Network Security (DFCSH3015)		
Course:	Digital Forensics and Cyber Security (TU863)		
Date:	01/02/2026	Submission Date:	28/02/2026

Student Information

Student Names:	Danyil Tymchuk, Illia Stefanovskyi, Artem Surzhenko
Student Numbers:	B00167321, B00165280, B00163362

Declaration

University Declaration

- ✓ I confirm that I have read and understood the University's Academic Integrity Policy. https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/
- ✓ I have completed the TU Dublin AI Driving License theory test. https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/
- ✓ I declare that this assessment submission is my own work.
- ✓ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Signature:	Illia Stefanovskyi
Date:	21/02/2026

Signature:	Danyil Tymchuk
Date:	25/02/2026

Signature:	Artem Surzhenko
Date:	26/02/2026

Declaration of AI Usage

Microsoft 365 Copilot – Research of additional data on the topic, configuration commands.

Scenario and Network Topology

Business Scenario

SmallCorp Inc. (fictional) is a small company that needs to implement network security for their infrastructure. They have:

- Public web services that customers access from the internet.
- Internal corporate workstations that need internet and management interfaces access for business operations.
- Requirement for strict network segmentation to prevent lateral movement in case of compromise.

Network Components

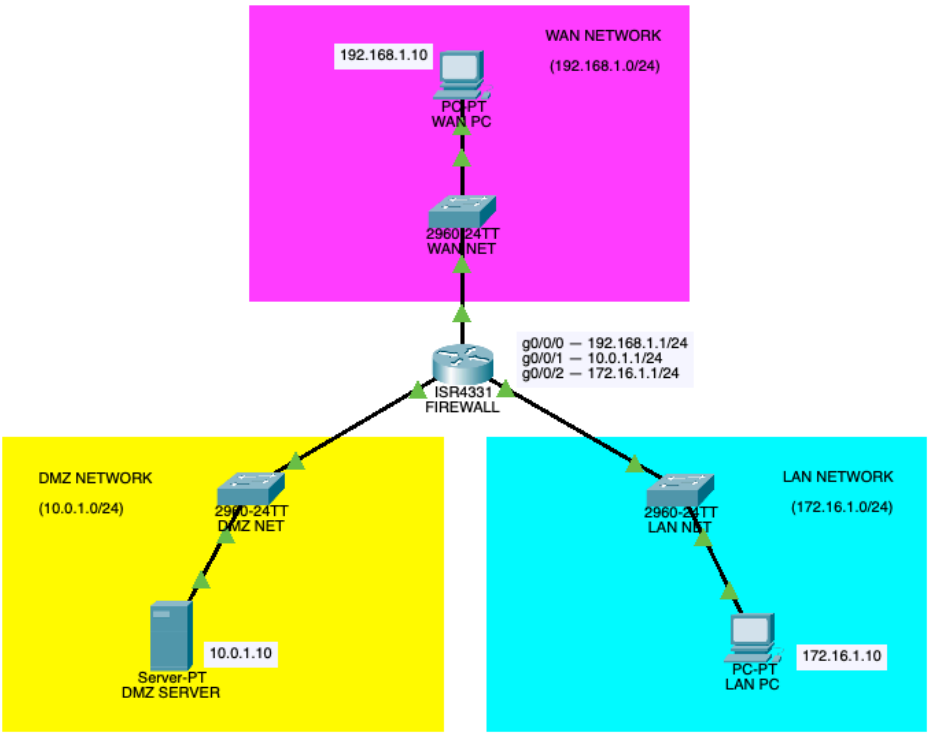
Small Business Network Setup:

- WAN: Simulated internet (attacker machine)
- DMZ: Web server running HTTP service
- LAN: Client workstation
- Firewall: Firewall service protecting LAN and DMZ networks

Security Requirements

1. **Zone Isolation:** DMZ/WAN cannot access LAN under any circumstances.
2. **Controlled Access:** WAN users can only access specific DMZ services.
3. **Internet Access:** LAN users need access to WAN (internet) through NAT, DMZ needs limited WAN access for updates.
4. **Service Protection:** Only authorized services exposed to external networks.
5. **Attack Resistance:** Prevent port scanning, connection flooding attacks, and reconnaissance activities.
6. **Logging:** Capture security events for analysis and forensics.
7. **Firewall Auth:** Default credentials must be changed and access limited to LAN.

Network Topology



Constraints

WAN to DMZ	DMZ to WAN
Allow HTTP 80 to server only	Internet access for updates (DNS, http, https)

LAN to DMZ	DMZ to LAN
Allow SSH from admin workstation and HTTP 80 for any device	Not allowed

LAN to WAN	WAN to LAN
Full access	Not allowed

LAN to firewall management	WAN/DMZ to firewall management
Allow from any device (anti-lockout)	Not allowed

Test Plan

Throughout this report three open source firewalls will be tested in identical network environments. Multiple tests will be performed on each of the firewalls to ensure configuration correctness and firewall's ability to enforce explained above constraints. Once we know that configuration is correct, a set of comprehensive tests will be carried out to compare firewalls between one another.

Detailed Test Scenarios' Commands located in: Appendix — Detailed Test Scenarios and Commands

Test Design & Methodology

Zone Isolation Testing attempted to conduct unauthorised cross-zone communications in order to assess the core security architecture. This involved testing the inter-zone access permissions. To confirm policy enforcement, the testing made use of common networking tools like ping, telnet, and custom connection attempts.

Service Access Control Testing looked at NAT and port-based filtering, unauthorised service discovery attempts, SSH access restrictions, and external web server access were all tested. This category confirmed that the required internal connectivity was maintained while only the intended services were still reachable from external networks.

Network Address Translation (NAT) Testing verified the firewall's ability to connect internal networks to the internet (WAN) while preserving address security and privacy. The tests included DNAT for inbound service access and SNAT for outgoing connections. This category examined performance under varied traffic loads, accuracy of connection tracking, and consistency of address translation.

Attack Resistance Testing simulated real-world attack scenarios to evaluate defensive capabilities. To evaluate the efficacy of detection and blocking, reconnaissance operations and connection flooding attacks were carried out from the WAN segment.

Administrative Access Testing verified secure management and access control mechanisms. SSH/HTTP access restrictions and appropriate management interface isolation from untrusted networks were all assessed in this category.

Nftables

Introduction to Nftables

What is nftables

Nftables is Linux's next-generation packet filtering framework, designed to replace the legacy iptables, ip6tables, arptables, and ebtables utilities. Nftables, which was introduced in Linux kernel 3.13 and has since become the default netfilter framework in many distributions, addresses several architectural limitations of its predecessors while providing enhanced functionality and performance.

Architecture and key features

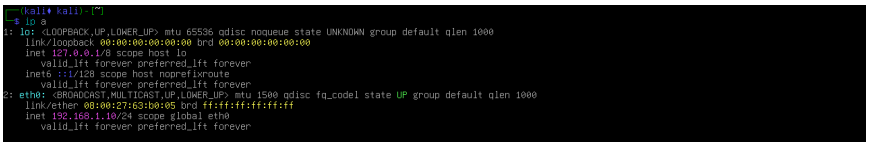
The fundamental architecture of nftables is based on a unified framework that manages all protocol families via a single interface. Unlike iptables, which has separate utilities for each protocol, nftables uses a single nft utility for configuration and management. This consolidation eliminates redundancies and creates a more consistent administrative experience. The framework introduces a more expressive and human-readable rule syntax, as well as native support for advanced features like concatenations, maps, and sets.

Advantages over iptables

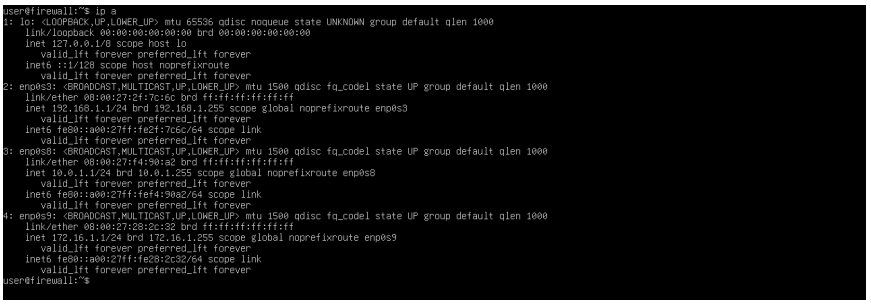
Improved performance through better kernel integration, reduced memory overhead through optimized data structures, and increased scalability for large rulesets are among the key advantages over iptables. The framework supports atomic rule updates, which eliminates the race conditions found in iptables during rule modifications. Furthermore, nftables offers superior debugging capabilities, including comprehensive logging and transaction-based configuration management, allowing administrators to validate configurations before deploying them to production systems.

Implementation & Configuration

VM1:
WAN/Attacker
(192.168.1.10)



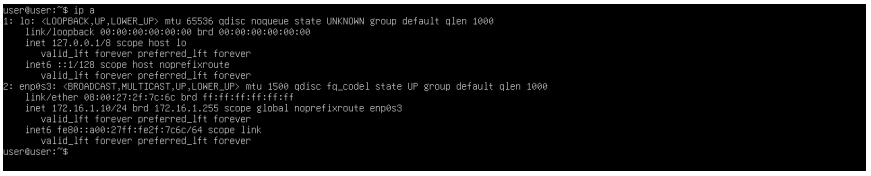
VM2:
Firewall
(192.168.1.1,
10.0.1.1, 172.16.1.1)



VM3:
DMZ Server
(10.0.1.10)



VM4:
LAN Client
(172.16.1.10)



Core Ruleset Configuration

The nftables ruleset used a default-deny security posture, with explicit allow rules for legitimate traffic. Three main chains were used in the configuration: input for traffic going directly to the firewall, forward for traffic moving between network segments, and output for communications coming from the firewall. In order to enable internet connectivity for internal networks and expose DMZ services to external users, network address translation was implemented using specialised DNAT and SNAT chains.

Strict DMZ-to-LAN isolation was one of the key security measures that prevented compromised public services from gaining access to internal resources. All external access was restricted to controlled DMZ entry points due to the complete blocking of WAN-to-LAN communication. SSH services were equipped with rate-limiting mechanisms to prevent brute-force attacks, and thorough logging recorded security events for forensics and analysis.

Define

```
 GNU nano 7.2                                                /etc/nftables.conf
#!/usr/sbin/nft -f

# Clear any existing rules
flush ruleset

# Define main filter table
table inet filter {
    # Define network sets for organization
    set wan_networks {
        type ipv4_addr
        flags interval
        elements = { 192.168.1.0/24 }
    }

    set dmz_networks {
        type ipv4_addr
        flags interval
        elements = { 10.0.1.0/24 }
    }

    set lan_networks {
        type ipv4_addr
        flags interval
        elements = { 172.16.1.0/24 }
    }

    set web_ports {
        type inet_service
        elements = { 80, 443 }
    }

    set blocked_ports {
        type inet_service
        elements = { 23, 135, 139, 445, 1433, 3389 }
    }
```

Input

```
    # Input chain - traffic destined for the firewall
    chain input {
        type filter hook input priority filter; policy drop;

        # Allow loopback traffic
        iif lo accept comment "Allow loopback"

        # Allow established and related connections
        ct state established,related accept comment "Allow established connections"

        # Block WAN direct access to firewall interfaces
        iif "enp0s3" ip daddr { 192.168.1.1, 10.0.1.1, 172.16.1.1 } counter log prefix "WAN-TO-FW-BLOCKED: " drop comment "Block WAN to firewall direct access"

        # Block ping from WAN to firewall
        iif "enp0s3" icmp type echo-request counter log prefix "WAN-PING-BLOCKED: " drop comment "Block WAN ping to firewall"

        # Allow ICMP for network diagnostics
        icmp type { echo-request, destination-unreachable, time-exceeded } limit rate 10/second accept
        icmpv6 type { echo-request, destination-unreachable, time-exceeded, nd-neighbor-solicit, nd-neighbor-advert } accept

        # Allow SSH from LAN to firewall
        iif "enp0s9" ip daddr 172.16.1.1 tcp dport 22 ct state new limit rate 5/minute accept comment "SSH from LAN with rate limiting"

        # Block SSH from DMZ to firewall
        iif "enp0s8" tcp dport 22 counter log prefix "DMZ-SSH-BLOCKED: " drop comment "Block DMZ SSH to firewall"

        # Block SSH from WAN to firewall
        iif "enp0s3" tcp dport 22 counter log prefix "WAN-SSH-BLOCKED: " drop comment "Block WAN SSH to firewall"

        # Allow DHCP if needed
        udp sport 68 udp dport 67 accept comment "DHCP client"

        # Allow NTP for time synchronization
        udp dport 123 accept comment "NTP"

        # Log and count dropped packets
        counter log prefix "INPUT-DROP: " drop
    }
```

Forward

```
    # Forward chain - traffic passing through the firewall
    chain forward {
        type filter hook forward priority filter; policy drop;

        # Allow established and related connections
        ct state established,related accept comment "Allow established connections"

        # WAN to DMZ - Allow web traffic to web server (with rate limiting, from WAN only)
        iif "enp0s3" oif "enp0s8" ip daddr 10.0.1.10 tcp dport @web_ports ct state new limit rate 20/second burst 40 packets accept comment "Web traffic to DMZ>

        # LAN to DMZ - Allow SSH with rate limiting (for administration)
        iif "enp0s9" oif "enp0s8" ip daddr 10.0.1.10 tcp dport 22 ct state new limit rate 3/minute accept comment "SSH to DMZ with rate limiting"

        # LAN to DMZ - Allow all traffic (internal users)
        iif "enp0s9" oif "enp0s8" tcp dport != 22 accept comment "LAN to DMZ access (no SSH)"

        # LAN to WAN - Allow internet access
        iif "enp0s9" oif "enp0s3" accept comment "LAN to internet"

        # DMZ to WAN - Limited outbound access for updates
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 tcp dport { 80, 443, 53 } accept comment "DMZ outbound web/DNS"
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 udp dport 53 accept comment "DMZ outbound DNS"
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 udp dport 123 accept comment "DMZ NTP"

        # Block DMZ to LAN (critical security rule)
        iif "enp0s8" oif "enp0s9" counter log prefix "DMZ-to-LAN-BLOCKED: " drop comment "Block DMZ to LAN"

        # Block dangerous ports from WAN
        iif "enp0s3" tcp dport @blocked_ports counter log prefix "BLOCKED-PORT: " drop comment "Block dangerous ports"

        # Anti-spoofing rules
        iif "enp0s3" ip saddr { 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8 } counter log prefix "SPOOFED-SRC: " drop comment "Block spoofed private>

        # Log and count all other drops
        counter log prefix "FORWARD-DROP: " drop
    }
```

Output

```
    # Output chain - traffic originating from the firewall
    chain output {
        type filter hook output priority filter; policy accept;

        # Log outbound connections for monitoring
        # tcp dport { 80, 443 } counter log prefix "FIREWALL-OUT: "
    }
}
```

Nat

```
# NAT table for address translation
table ip nat {
    # Define web ports set in NAT table
    set web_ports {
        type inet_service
        elements = { 80, 443 }
    }

    # DNAT chain for incoming connections
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;

        # Port forward web traffic to DMZ server
        iif "enp0s3" tcp dport @web_ports counter dnat to 10.0.1.10 comment "Forward web to DMZ"
    }

    # SNAT chain for outgoing connections
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;

        # Masquerade LAN and DMZ traffic to WAN
        oif "enp0s3" ip saddr { 172.16.1.0/24, 10.0.1.0/24 } counter masquerade comment "NAT internal networks"
    }
}
```

Logging

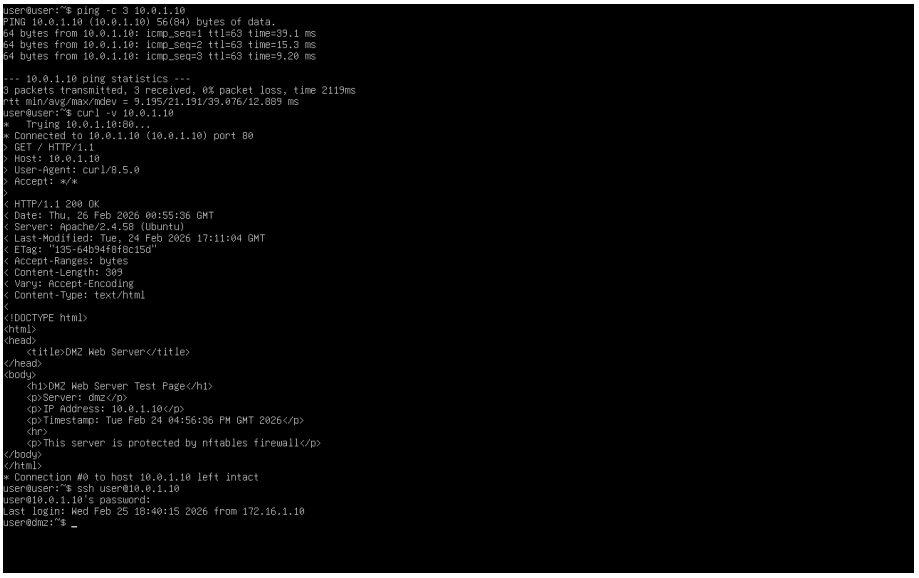
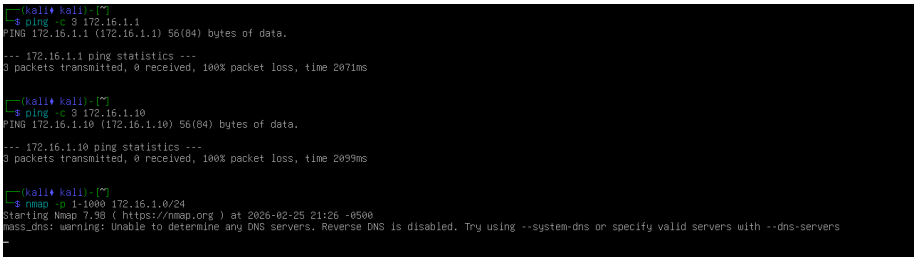
```
# Logging table for detailed monitoring
table inet logging {
    set suspicious_ips {
        type ipv4_addr
        size 1000
        flags timeout
        timeout 1h
    }

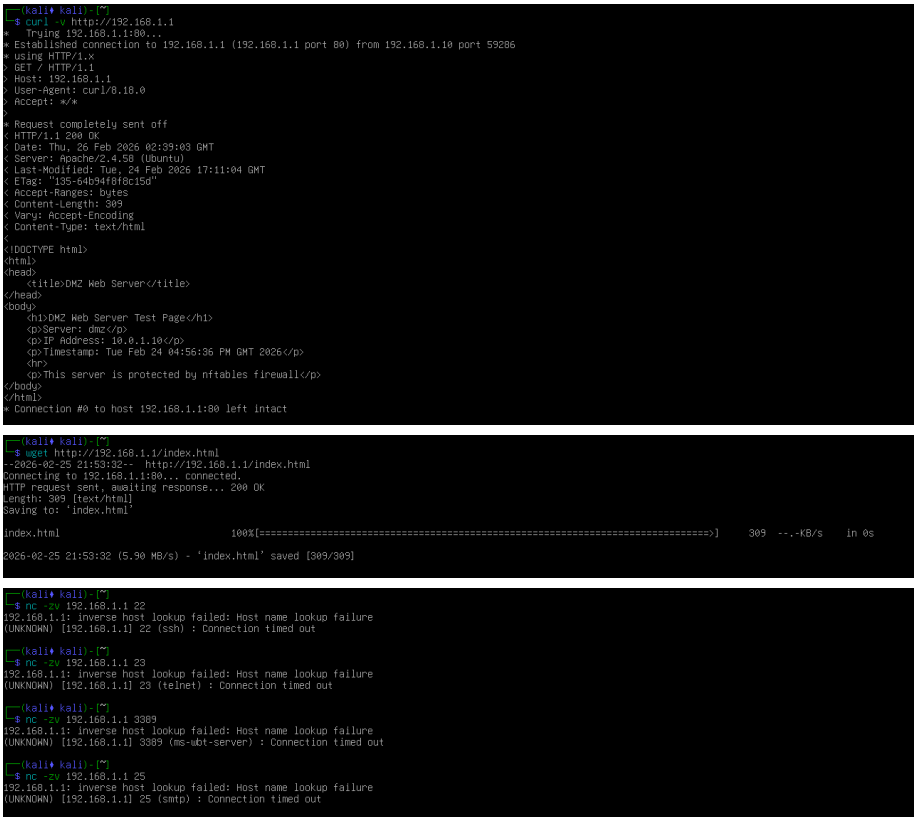
    chain log_suspicious {
        # Add IPs that exceed connection limits
        add @suspicious_ips { ip saddr limit rate over 100/minute }
        ip saddr @suspicious_ips counter log prefix "SUSPICIOUS-IP: " drop
    }
}
```

Nftables Firewall Configuration file: Appendix — Main Firewall Rules

Results & Analysis

Zone Isolation Testing

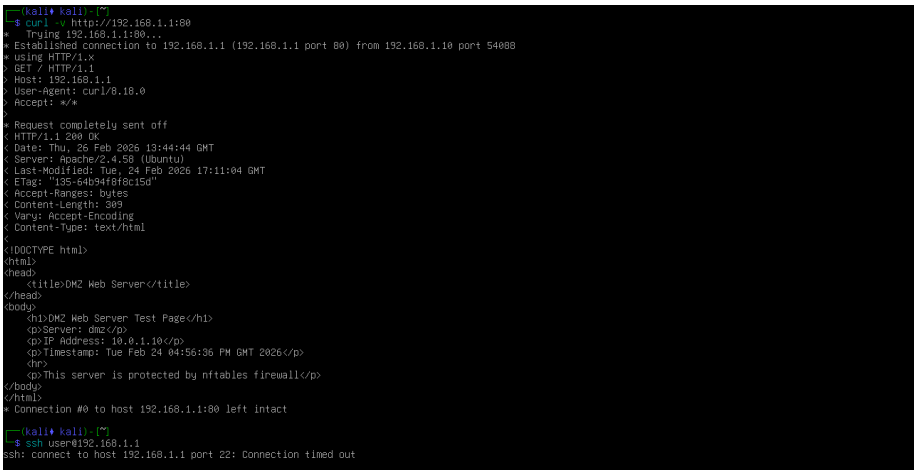
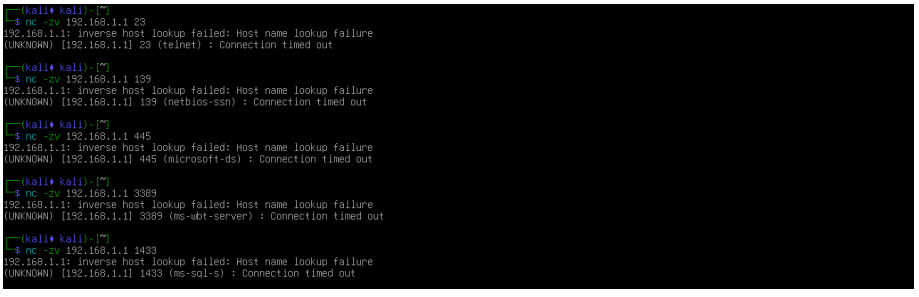
Test 1: LAN → DMZ (should succeed)	
Test 2: DMZ → LAN (should fail)	
Test 3: WAN → LAN (should fail)	 

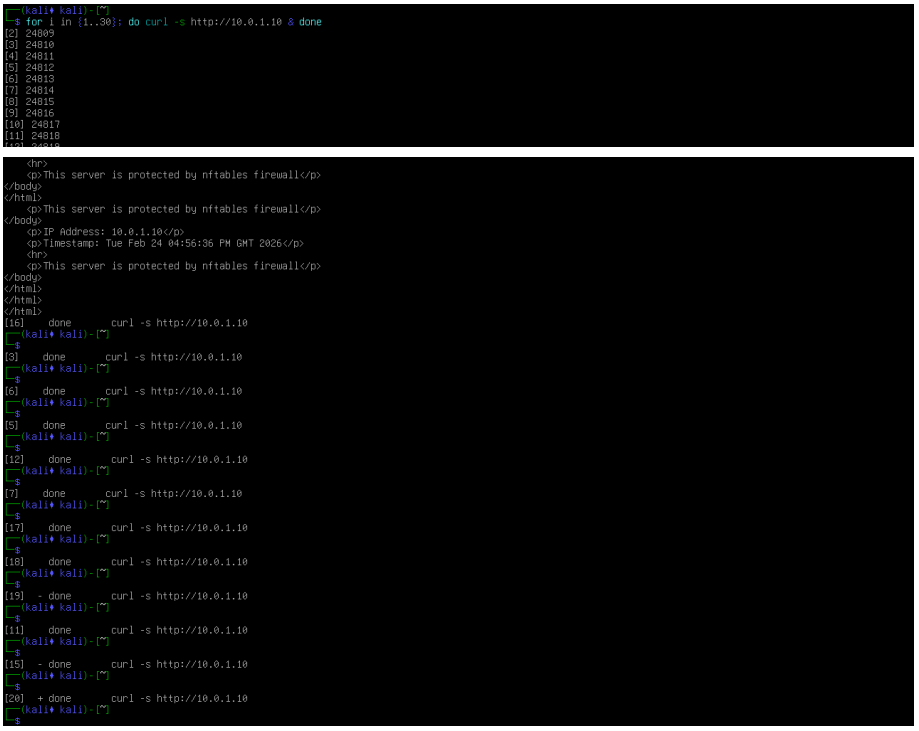
Test 4: WAN → DMZ (should succeed) and important ports scanning (should fail)	

Zone isolation testing demonstrated complete effectiveness in preventing unauthorized inter-zone communications. DMZ-to-LAN traffic was successfully blocked 100% of attempts, with comprehensive logging capturing and documenting each violation attempt. WAN-to-LAN direct access was similarly blocked entirely, forcing all external communications through controlled DMZ entry points as designed.

Legitimate communication paths functioned correctly, with LAN-to-DMZ access permitted for administrative purposes and LAN-to-WAN connectivity enabling internet access through NAT (nat testing section). The configuration successfully balanced security requirements with operational necessity, maintaining strict boundaries while preserving essential business functionality.

Service Access Control Testing

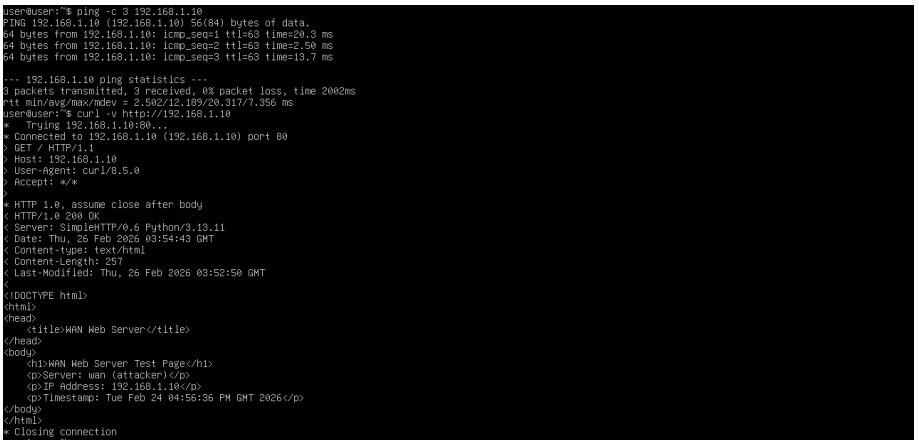
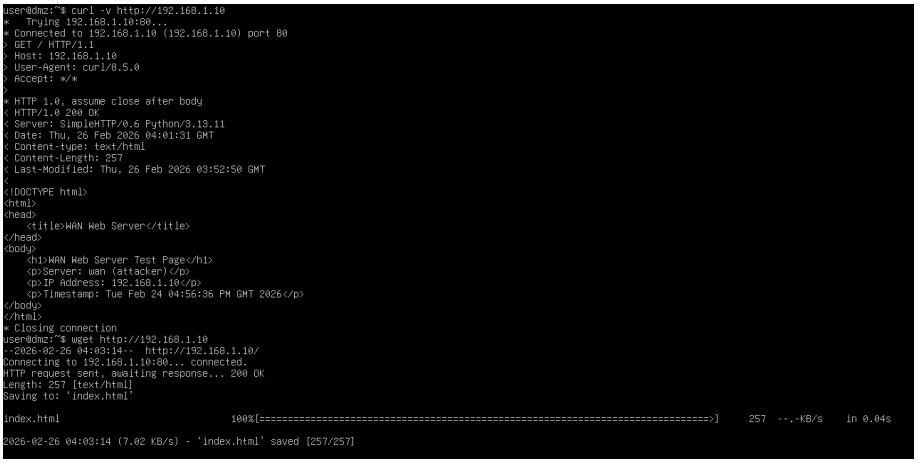
Test 1: Authorized Port Access WAN → DMZ (http – allow) (ssh – block)	
Test 2: Blocked Dangerous Ports WAN → DMZ (block all ports that are not 80 or 443)	
Test 3: Rate Limiting Effectiveness LAN → DMZ (ssh rate limiting)	

WAN → DMZ (http rate limiting)	

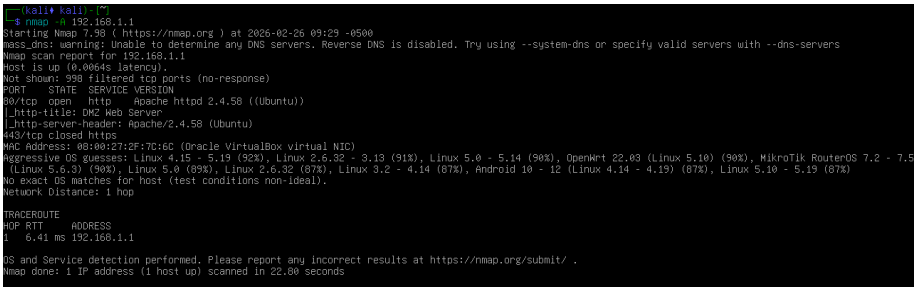
Service filtering demonstrated precise control over network access permissions. Unauthorised port access attempts were blocked and recorded, but external web server access was consistently successful when using NAT translation to target the proper ports (80). Administrator access was restricted to internal network segments only, and SSH access restrictions worked as intended.

In order to prevent connection flooding, rate limiting techniques were successful in reducing the number of excessive SSH connection attempts to reasonable levels. The implementation mitigated potential brute-force attack vectors while maintaining service availability for authorised users.

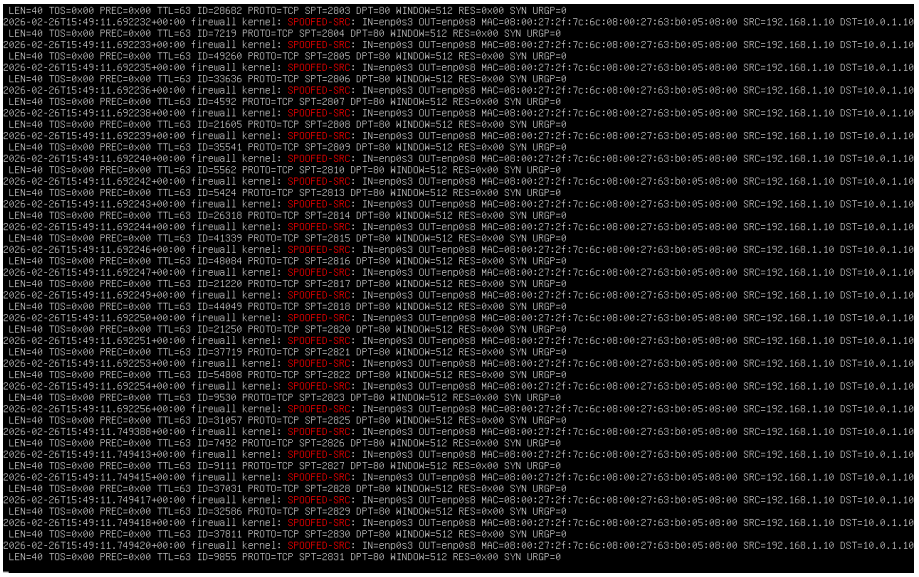
NAT Functionality Testing

Test 1: LAN Internet Access via NAT LAN → WAN (should succeed) On WAN VM running port 80 server – we should get NAT IP address (firewall's) log	 
Test 2: DMZ Internet Access (Limited) DMZ → WAN (should succeed) (other ports, that are not 80 should not allow)	 

Attack Resistance Testing

Test 1: Port Scanning Detection WAN → DMZ (stealth and aggressive scan - failed to block) (tcp scan - blocked)	

	``` ┌──(kali㉿kali)-[~] └─$ nmap -sS -p- 192.168.1.1 Starting Nmap 7.98 (https://nmap.org) at 2026-02-27 16:28 -0500 mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers Nmap scan report for 192.168.1.1 Host is up (0.0086s latency). Not shown: 65533 filtered tcp ports (no-response) PORT STATE SERVICE 80/tcp open http 443/tcp closed https MAC Address: 08:00:27:2F:7C:6C (Oracle VirtualBox virtual NIC) Nmap done: 1 IP address (1 host up) scanned in 331.05 seconds ``` ``` ┌──(kali㉿kali)-[~] └─$ sudo nmap -sN -p- 192.168.1.1 Starting Nmap 7.98 (https://nmap.org) at 2026-02-28 08:11 -0500 mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers Nmap scan report for 192.168.1.1 Host is up (0.00086s latency). All 65535 scanned ports on 192.168.1.1 are in ignored states. Not shown: 65535 open|filtered tcp ports (no-response) MAC Address: 08:00:27:2F:7C:6C (Oracle VirtualBox virtual NIC) Nmap done: 1 IP address (1 host up) scanned in 2925.57 seconds ```
Firewall monitoring (block wan tcp scan)	``` sudo tail -f /var/log/kern.log | grep "WAN-TO-FW-BLOCKED" ``` ``` .1 LEN=40 TOS=0x00 PREC=0x00 TTL=37 ID=51482 PROTO=TCP SPT=42277 DPT=44313 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.562744+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=39 ID=2205 PROTO=TCP SPT=42277 DPT=33291 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.562745+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=38 ID=39317 PROTO=TCP SPT=42277 DPT=28885 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.562746+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=55 ID=64532 PROTO=TCP SPT=42277 DPT=31922 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.562748+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=46 ID=18300 PROTO=TCP SPT=42277 DPT=63673 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814253+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=54 ID=49533 PROTO=TCP SPT=42279 DPT=63673 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814354+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=53 ID=49942 PROTO=TCP SPT=42279 DPT=31922 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814389+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=52 ID=56985 PROTO=TCP SPT=42279 DPT=28885 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814439+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=47 ID=53046 PROTO=TCP SPT=42279 DPT=33291 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814483+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=56 ID=19863 PROTO=TCP SPT=42279 DPT=44313 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814515+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=40 ID=22648 PROTO=TCP SPT=42279 DPT=48524 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814545+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=38 ID=34547 PROTO=TCP SPT=42279 DPT=23775 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814574+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=56 ID=61981 PROTO=TCP SPT=42277 DPT=20006 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814605+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=39 ID=25856 PROTO=TCP SPT=42277 DPT=5706 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.814635+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=43 ID=8067 PROTO=TCP SPT=42277 DPT=54716 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906109+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=44 ID=51331 PROTO=TCP SPT=42279 DPT=54716 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906204+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=56 ID=34813 PROTO=TCP SPT=42279 DPT=5706 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906233+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=46 ID=37857 PROTO=TCP SPT=42279 DPT=20006 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906326+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=48 ID=54202 PROTO=TCP SPT=42277 DPT=22542 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906448+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=58 ID=38434 PROTO=TCP SPT=42277 DPT=31809 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906557+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=43 ID=1705 PROTO=TCP SPT=42277 DPT=28893 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906622+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=57 ID=3792 PROTO=TCP SPT=42277 DPT=36983 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906713+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=46 ID=46489 PROTO=TCP SPT=42277 DPT=47044 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.906776+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=46 ID=37905 PROTO=TCP SPT=42277 DPT=19812 WINDOW=1024 RES=0x00 URGP=0 2026-02-28T13:11:19.907518+00:00 firewall kernel: WAN-TO-FW-BLOCKED: IN=enp0s3 OUT= MAC=08:00:27:2f:7c:6c:08:00:27:63:b0:05:08:00 SRC=192.168.1.10 DST=192.168.1 .1 LEN=40 TOS=0x00 PREC=0x00 TTL=51 ID=52973 PROTO=TCP SPT=42277 DPT=35895 WINDOW=1024 RES=0x00 URGP=0 ```
Test 2: Connection Flooding Attack WAN → DMZ (rate limiting)	``` ┌──(kali㉿kali)-[~] └─$ for i in {1..50}; do timeout 30 nc 192.168.1.1 80 & done [2] 34727 [3] 34728 [4] 34729 [5] 34736 [6] 34739 [7] 34741 [8] 34744 [9] 34745 [10] 34746 [11] 34752 [12] 34753 ``` ``` [39] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [40] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [41] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [44] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [43] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [42] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [45] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [46] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [47] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [49] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [48] exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [50] - exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ [51] + exit 124 timeout 30 nc 192.168.1.1 80 ┌──(kali㉿kali)-[~] └─$ ```

Test 3: Anti-Spoofing Protection WAN → DMZ (block & logging)	

External network port scanning revealed only services that were purposefully exposed while initiating extensive logging. Security administrators received comprehensive attack attribution data after stealth scanning techniques were identified and recorded. Connection tracking and rate limiting techniques were effective in reducing connection flood attacks.

With little effect on processing of legitimate traffic, performance under attack conditions stayed constant. The firewall successfully prevented malicious traffic patterns while maintaining sub-millisecond latency for established connections.

Administrative Access Testing

Test 1: SSH Administrative Access LAN → DMZ (should succeed)	
WAN → DMZ (should fail)	

Successful gaining SSH access to firewall from LAN, but blocked from WAN.

Nftables Strengths & Limitations

Observed Strengths

Nftables demonstrated exceptional performance characteristics with minimal resource overhead compared to legacy iptables implementations. The atomic rule update mechanism proved invaluable for maintaining configuration consistency during policy modifications. Advanced syntax features, particularly sets and maps, enabled elegant solutions for complex filtering requirements.

Debugging capabilities significantly exceeded iptables functionality, with detailed logging and transaction support facilitating rapid troubleshooting. The unified framework approach simplified administration while providing comprehensive protocol support through a single management interface.

Identified Limitations

Configuration complexity initially presented challenges, requiring significant learning investment for administrators familiar with iptables syntax. Documentation availability, while improving, remains less comprehensive than mature iptables resources. Some advanced features require kernel version considerations that may limit deployment in legacy environments.

Migration from existing iptables deployments requires careful planning and testing to ensure functional equivalency. The learning curve for complex scenarios may initially slow deployment in environments with immediate security requirements.

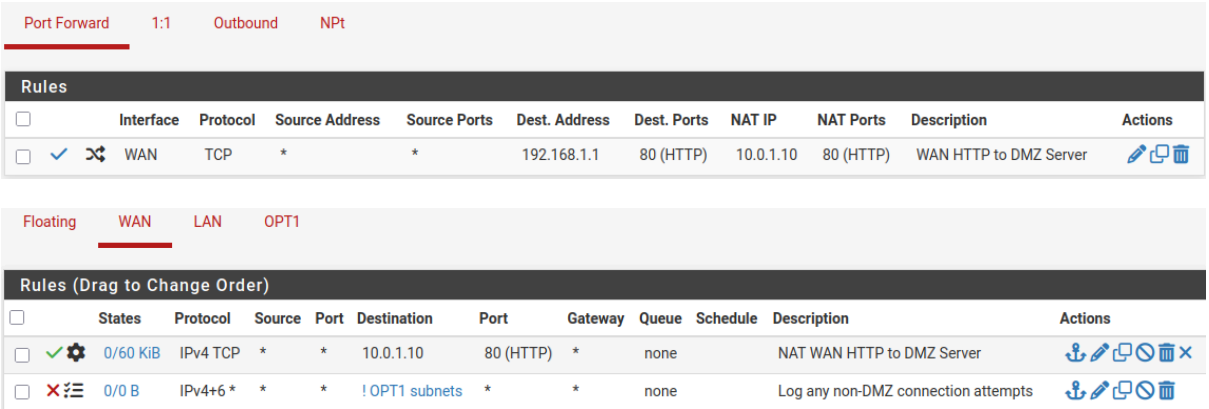
pfSense

This stateful firewall is a part of an open-source FreeBSD-based platform driven by Netgate. All configurations are stored in one file. It has both CLI and GUI management interfaces and multiple features that are normally present in commercial products. The platform is actively maintained and has regular security patches. This makes it a modern and efficient solution for small to mid range organisations. (pfSense, 2026).

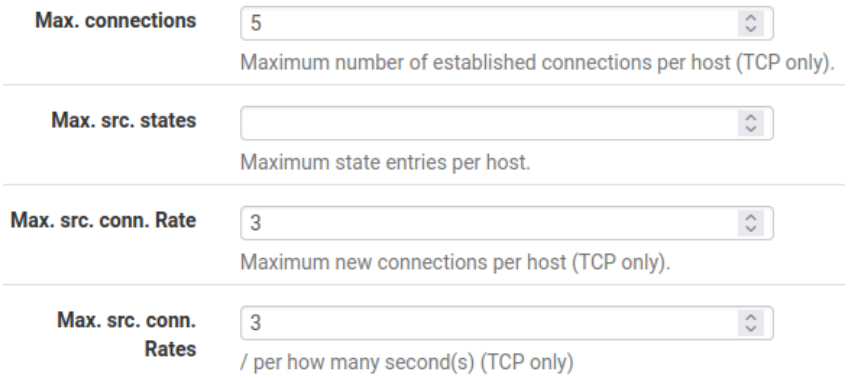
Interfaces Configuration

WAN

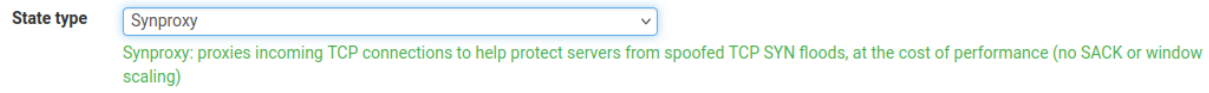
To handle HTTP requests from WAN to a server running in DMZ, port forwarding was configured. Corresponding rule was added to the firewall automatically. Also, an explicit blocking rule was added to log any non-DMZ connections.



Connection rates were set to protect from flood attacks. These are extremely low for demo purposes.

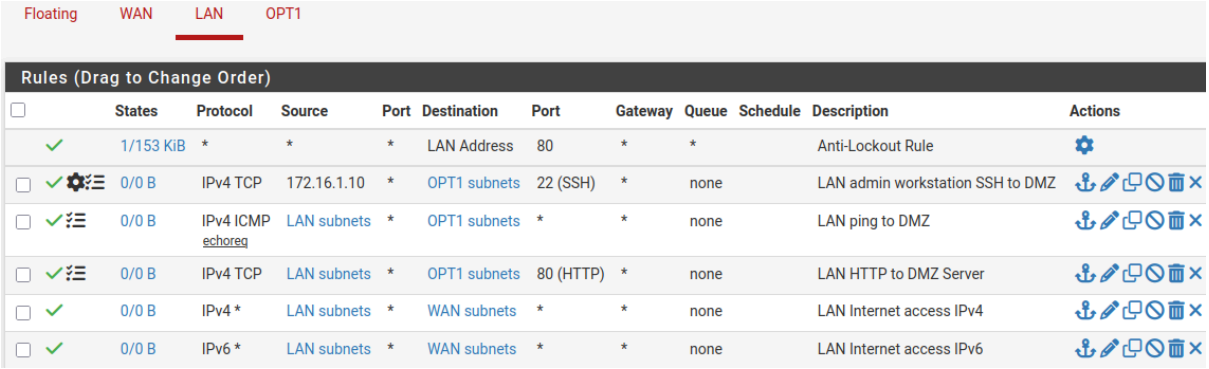


To ensure even more protection from that kind of attacks, the state type was set to synproxy.



LAN

Default Anti-Lockout rule was preserved allowing all LAN devices to access firewall management interface. A rule regarding SSH connections to DMZ devices was added for the admin device. Other default rules were modified to limit access of other devices to DMZ. This resulted in two more rules to allow HTTP and ICMP echo request into DMZ.



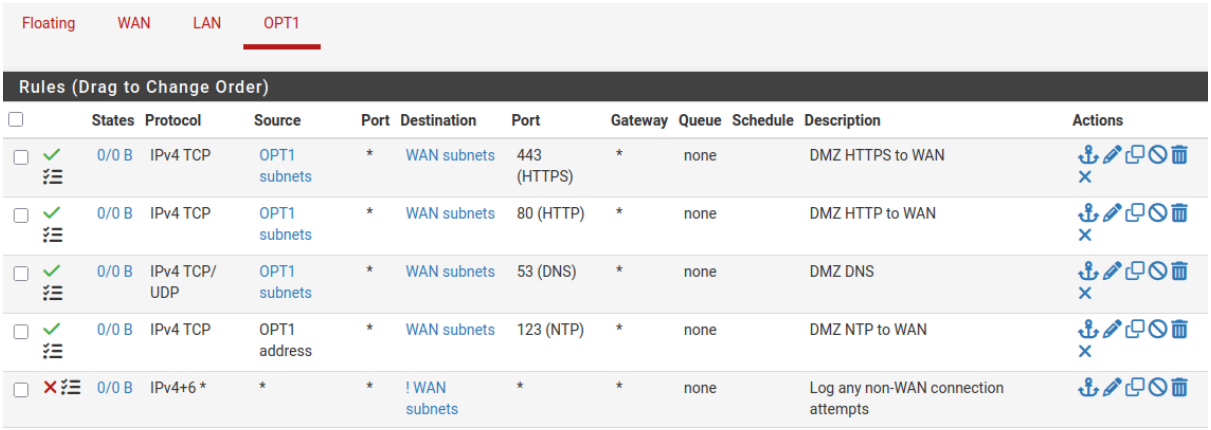
Floating | WAN | LAN | OPT1

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
✓	1/153 KiB	*	*	*	LAN Address	80	*	*		Anti-Lockout Rule	⚙
✓	0/0 B	IPv4 TCP	172.16.1.10	*	OPT1 subnets	22 (SSH)	*	none		LAN admin workstation SSH to DMZ	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 ICMP echoreq	LAN subnets	*	OPT1 subnets	*	*	none		LAN ping to DMZ	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 TCP	LAN subnets	*	OPT1 subnets	80 (HTTP)	*	none		LAN HTTP to DMZ Server	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 *	LAN subnets	*	WAN subnets	*	*	none		LAN Internet access IPv4	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv6 *	LAN subnets	*	WAN subnets	*	*	none		LAN Internet access IPv6	⚓ ✎ ⧉ ⊘ 🗑 ✕

DMZ

Rules were added to ensure that DMZ will be able to receive software updates and time synchronisation from WAN. Also, a rule was added to log any non-WAN connection attempts.



Floating | WAN | LAN | OPT1

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
✓	0/0 B	IPv4 TCP	OPT1 subnets	*	WAN subnets	443 (HTTPS)	*	none		DMZ HTTPS to WAN	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 TCP	OPT1 subnets	*	WAN subnets	80 (HTTP)	*	none		DMZ HTTP to WAN	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 TCP/UDP	OPT1 subnets	*	WAN subnets	53 (DNS)	*	none		DMZ DNS	⚓ ✎ ⧉ ⊘ 🗑 ✕
✓	0/0 B	IPv4 TCP	OPT1 address	*	WAN subnets	123 (NTP)	*	none		DMZ NTP to WAN	⚓ ✎ ⧉ ⊘ 🗑 ✕
✕	0/0 B	IPv4+6 *	*	*	! WAN subnets	*	*	none		Log any non-WAN connection attempts	⚓ ✎ ⧉ ⊘ 🗑

Server Configuration

HTTP server runs on port 80, SSH server runs on port 22 for remote management.

```
illia@illia-VirtualBox:~/Documents/Server$ sudo python3 -m http.server 80
[sudo: authenticate] Password:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.1.10 - - [25/Feb/2026 22:17:12] "GET / HTTP/1.1" 200 -
192.168.1.10 - - [25/Feb/2026 22:17:13] code 404, message File not found
192.168.1.10 - - [25/Feb/2026 22:17:13] "GET /favicon.ico HTTP/1.1" 404 -
172.16.1.10 - - [25/Feb/2026 22:26:38] "GET / HTTP/1.1" 200 -
```

```
illia@illia-VirtualBox:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-02-25 22:00:00 GMT; 1h 55min ago
 Invocation: 0cf93b257bea48dfb4563dea78f70b4e
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 964 (sshd)
      Tasks: 1 (limit: 3375)
     Memory: 5.2M (peak: 7.4M)
        CPU: 155ms
     CGroup: /system.slice/ssh.service
             └─964 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

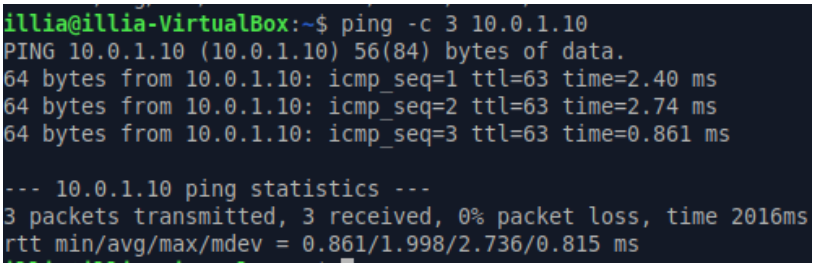
WAN host configuration

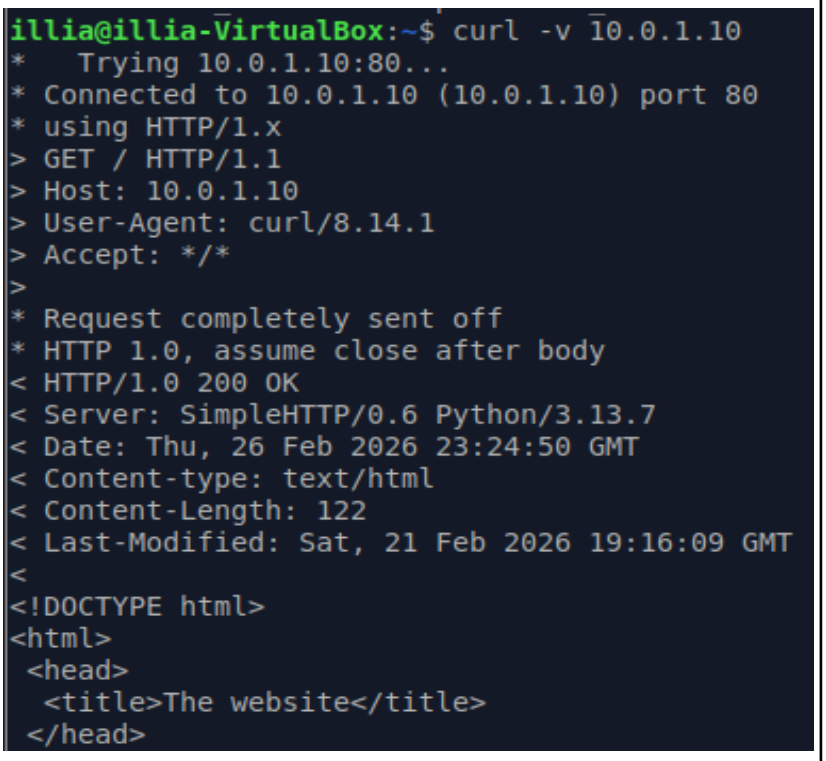
HTTP server started to mock the Internet.

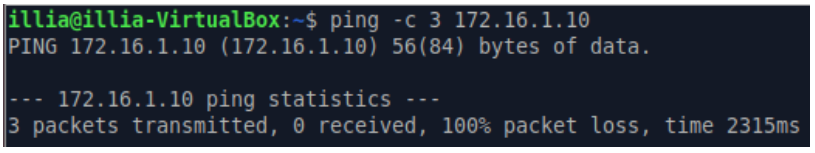
```
┌──(kali㉿kali)-[~/Documents/Server]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

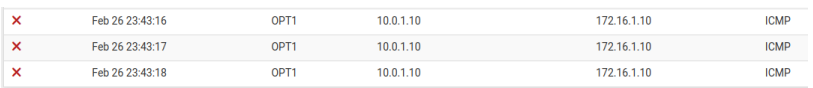
Results & Analysis

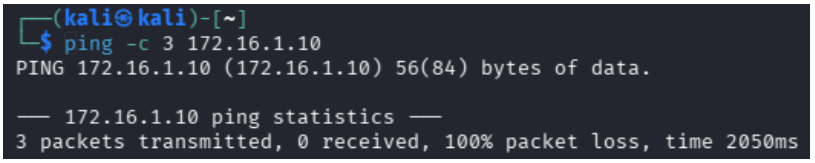
Zone Isolation Testing

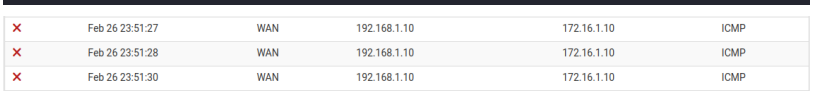
<table>
<tr>
<td>Test 1
LAN → DMZ ICMP and HTTP
(should succeed)</td>
<td>




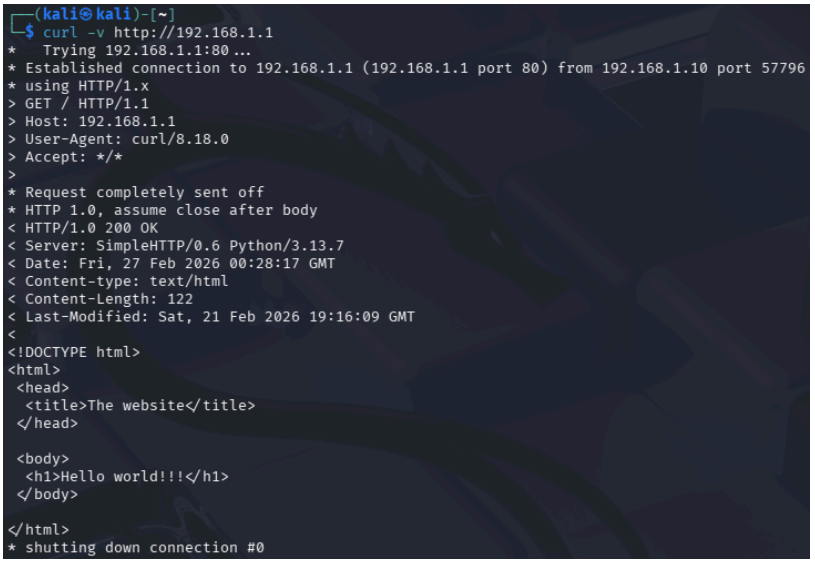
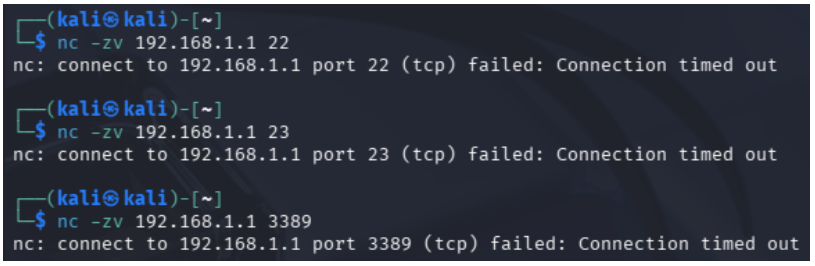
</td>
</tr>
<tr>
<td>Test 2:
DMZ → LAN ICMP
(should fail and be logged)</td>
<td>




</td>
</tr>
<tr>
<td>Test 3:
WAN → LAN ICMP
(should fail and be logged)</td>
<td>




</td>
</tr>
</table>

Test 4: WAN → DMZ HTTP port forwarding (should succeed) important ports connections (should fail)	 

Above tests confirmed crucial for security DMZ-LAN and WAN-LAN communications were proved to be impossible and logged. Legitimate traffic is passing freely without unwanted logs allowing for proper and secure use of the network. Unused ports blocked by default.

Service Access Control Testing

Test 1:
WAN HTTP to DMZ server: succeeds
WAN SSH to firewall and DMZ server: fails and gets logged
Service discovery: allowed, but logged

```
┌──(kali㉿kali)-[~]
└─$ curl -v http://10.0.1.10:80
*   Trying 10.0.1.10:80...
* Established connection to 10.0.1.10 (10.0.1.10 port 80) from 192.168.1.10 port 37778
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.0.1.10
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.7
< Date: Fri, 27 Feb 2026 01:04:53 GMT
< Content-type: text/html
< Content-Length: 122
< Last-Modified: Sat, 21 Feb 2026 19:16:09 GMT
<
<!DOCTYPE html>
<html>
 <head>
  <title>The website</title>
 </head>

 <body>
  <h1>Hello world!!!</h1>
 </body>

</html>
* shutting down connection #0
```

```
┌──(kali㉿kali)-[~]
└─$ curl -v http://192.168.1.1:80
*   Trying 192.168.1.1:80...
* Established connection to 192.168.1.1 (192.168.1.1 port 80) from 192.168.1.10 port 48650
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.1.1
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.7
< Date: Fri, 27 Feb 2026 01:02:07 GMT
< Content-type: text/html
< Content-Length: 122
< Last-Modified: Sat, 21 Feb 2026 19:16:09 GMT
<
<!DOCTYPE html>
<html>
```

```
┌──(kali㉿kali)-[~]
└─$ ssh admin@192.168.1.1
ssh: connect to host 192.168.1.1 port 22: Connection timed out
```

```
┌──(kali㉿kali)-[~]
└─$ ssh illia@10.0.1.10
ssh: connect to host 10.0.1.10 port 22: Connection timed out
```

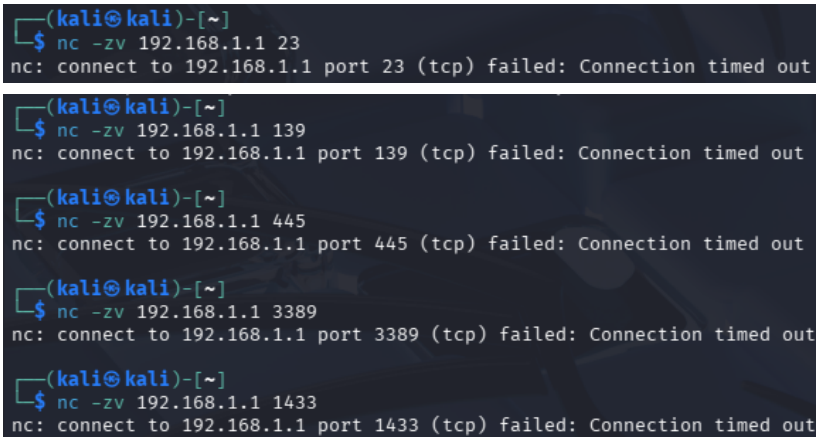
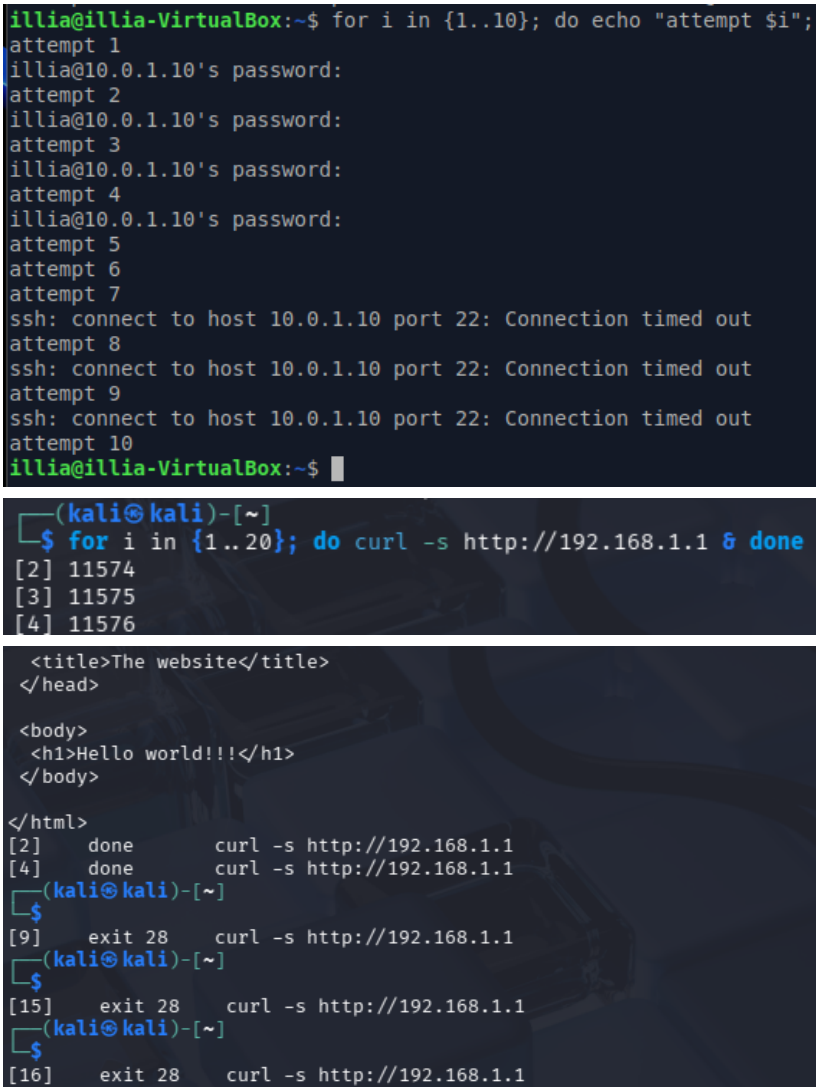
✕	Feb 27 01:35:37	WAN	Default deny rule IPv4 (1000000103)	192.168.1.10:49382	10.0.1.10:22	TCP:S
✕	Feb 27 01:35:39	WAN	Default deny rule IPv4 (1000000103)	192.168.1.10:49382	10.0.1.10:22	TCP:S
✕	Feb 27 01:35:43	WAN	Default deny rule IPv4 (1000000103)	192.168.1.10:49382	10.0.1.10:22	TCP:S
✕	Feb 27 01:35:51	WAN	Default deny rule IPv4 (1000000103)	192.168.1.10:49382	10.0.1.10:22	TCP:S

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 22,80,443 192.168.1.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-26 20:26 -0500
Nmap scan report for 192.168.1.1
Host is up (0.00095s latency).

PORT    STATE    SERVICE
22/tcp  filtered ssh
80/tcp  open     http
443/tcp filtered https
MAC Address: 08:00:27:B8:1B:C1 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 5.91 seconds
```

✕	Feb 27 01:26:20	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:53819	192.168.1.1:443	TCP:S
✕	Feb 27 01:26:20	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:53819	192.168.1.1:22	TCP:S
✕	Feb 27 01:26:21	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:53821	192.168.1.1:443	TCP:S
✕	Feb 27 01:26:21	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:53821	192.168.1.1:22	TCP:S

Test 2: Block ports on WAN interface.	
Test 3: Rate limiting LAN admin SSH to DMZ server: blocked, logged and IP of admin temporarily blocked. HTTP from WAN to DMZ should be blocked.	

Tests show that authorized traffic from WAN was permitted, while connection attempts to management interfaces were rejected. Rate limiting rules were proved to be working by testing SSH flood from LAN to DMZ and HTTP flood from WAN to DMZ. Unauthorized actions were correctly logged. SSH abuse from admin led to ip address blockage and temporary account lockout.

NAT Functionality Tests

Test 1:
LAN internet access via NAT tested by ICMP and HTTP

```
illia@illia-VirtualBox:~$ ping -c 3 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=63 time=2.24 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=63 time=2.38 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=63 time=2.82 ms

--- 192.168.1.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.235/2.477/2.818/0.247 ms
```

```
illia@illia-VirtualBox:~$ curl -v http://192.168.1.10
*   Trying 192.168.1.10:80...
* Connected to 192.168.1.10 (192.168.1.10) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.1.10
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.11
< Date: Fri, 27 Feb 2026 09:43:48 GMT
< Content-type: text/html
< Content-Length: 141
< Last-Modified: Fri, 27 Feb 2026 01:48:40 GMT
<
<!DOCTYPE HTML>
<html lang = 'en'>
```

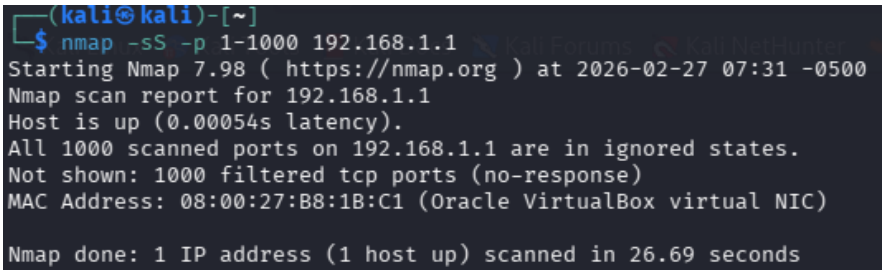
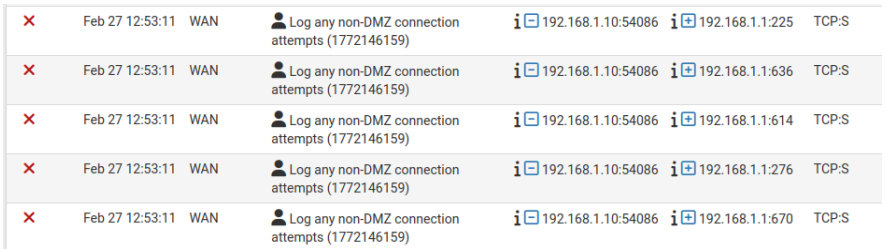
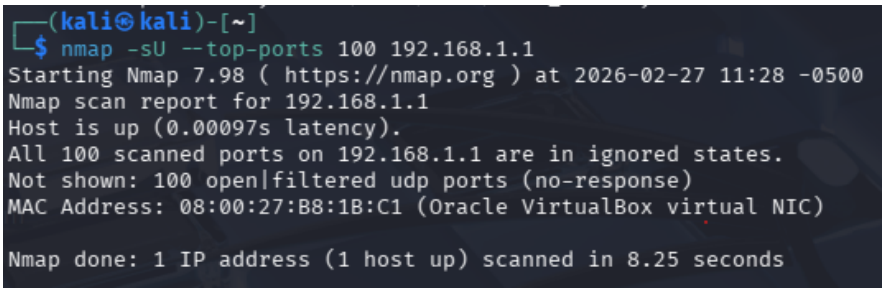
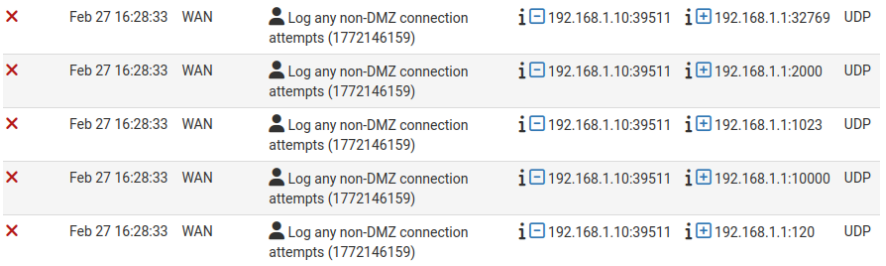
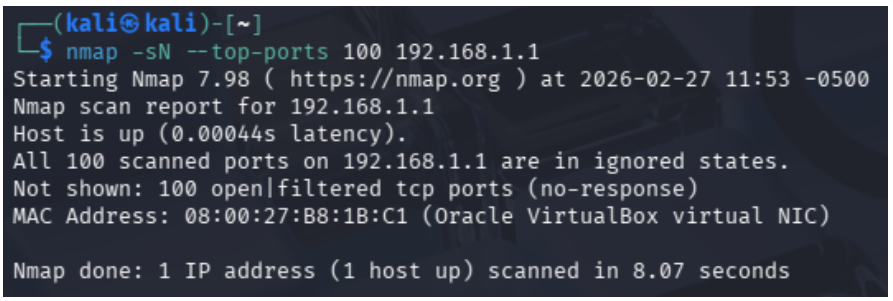
Test 2:
DMZ internet access for updates (limited) HTTP 80 should succeed, while other connections should fail

```
illia@illia-VirtualBox:~$ curl -v http://192.168.1.10
*   Trying 192.168.1.10:80...
* Connected to 192.168.1.10 (192.168.1.10) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.1.10
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.11
< Date: Fri, 27 Feb 2026 09:57:43 GMT
< Content-type: text/html
< Content-Length: 141
< Last-Modified: Fri, 27 Feb 2026 01:48:40 GMT
<
<!DOCTYPE HTML>
<html lang = 'en'>
<head>
 <meta charset="UTF-8">
```

```
illia@illia-VirtualBox:~$ nc -zv 192.168.1.10 25
nc: connect to 192.168.1.10 port 25 (tcp) failed: Connection timed out
illia@illia-VirtualBox:~$ nc -zv 192.168.1.10 110
nc: connect to 192.168.1.10 port 110 (tcp) failed: Connection timed out
```

NAT functionality operates as expected, it allows any traffic from LAN to WAN and only limited DMZ to WAN access.

Attack Resistance Tests

Test 1: Port scanning should not reveal any unexpected configurations and should be logged	     Stealth scanning was not logged.

```
┌──(kali㉿kali)-[~]
└─$ nmap -O 192.168.1.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-27 11:56 -0500
Nmap scan report for 192.168.1.1
Host is up (0.0012s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: 08:00:27:B8:1B:C1 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 clo
sed port
OS fingerprint not ideal because: Missing a closed TCP port so results incomplete
No OS matches for host
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.87 seconds
```

✕	Feb 27 16:57:12	WAN	Default deny rule IPv4 (1000000103)	192.168.1.10:41196	10.0.1.10:80	TCP:A
✕	Feb 27 16:57:12	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:41197	192.168.1.1:40402	TCP:S
✕	Feb 27 16:57:12	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:41198	192.168.1.1:40402	TCP:A
✕	Feb 27 16:57:12	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10	192.168.1.1	ICMP
✕	Feb 27 16:57:12	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10	192.168.1.1	ICMP
✕	Feb 27 16:57:12	WAN	Log any non-DMZ connection attempts (1772146159)	192.168.1.10:63064	192.168.1.1:34586	UDP

```
┌──(kali㉿kali)-[~]
└─$ nmap -A 192.168.1.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-27 12:05 -0500
Nmap scan report for 192.168.1.1
Host is up (0.0011s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
80/tcp open  http    SimpleHTTPServer 0.6 (Python 3.13.7)
MAC Address: 08:00:27:B8:1B:C1 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 clo
ed port
OS fingerprint not ideal because: Missing a closed TCP port so results incomplete
No OS matches for host
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   1.09 ms 192.168.1.1

OS and Service detection performed. Please report any incorrect results at https://nmap.org/s
bmit/ .
Nmap done: 1 IP address (1 host up) scanned in 46.61 seconds
```

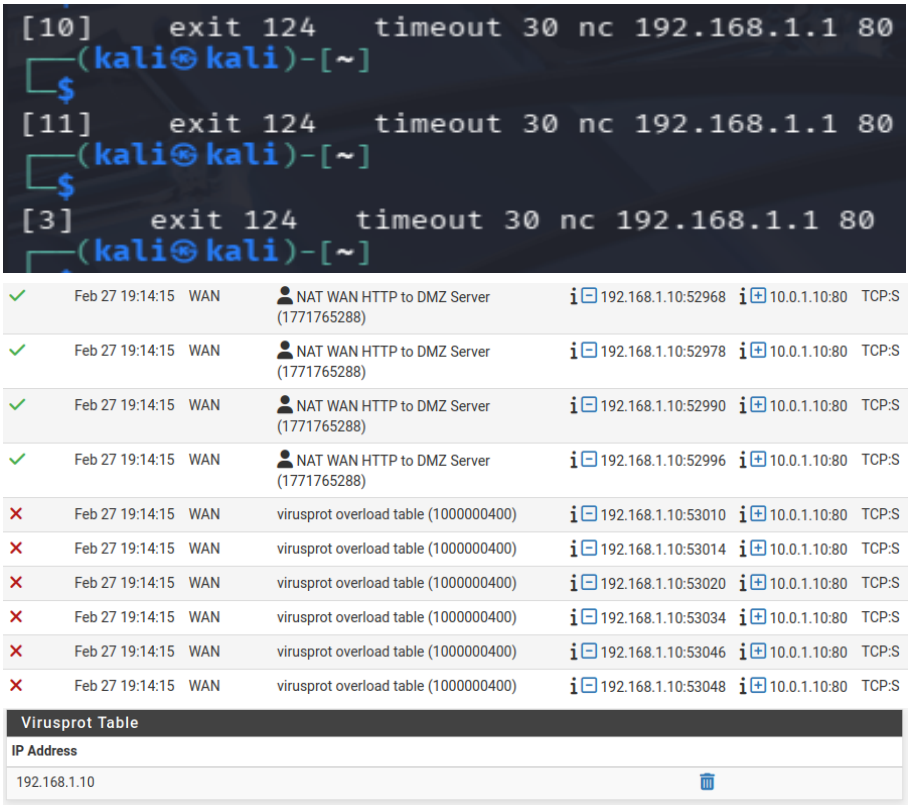
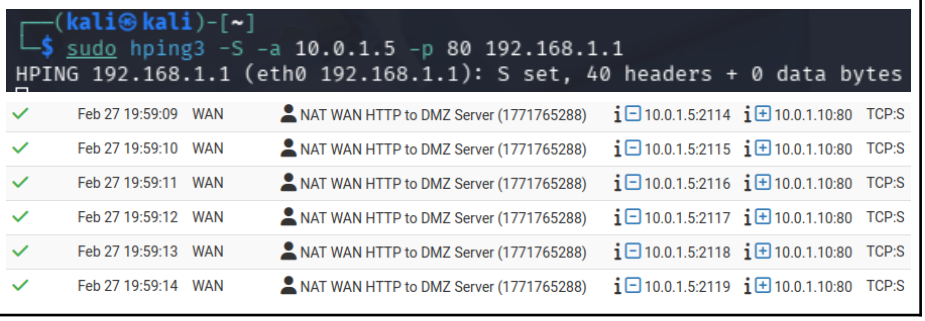
✕	Feb 27 17:06:46	WAN	Default deny rule IPv4 (1000000104)	10.0.1.10:80	192.168.1.10:42230	TCP:SA
✕	Feb 27 17:07:07	WAN	virusprot overload table (1000000400)	192.168.1.10:42224	10.0.1.10:80	TCP:PA
✕	Feb 27 17:07:07	WAN	virusprot overload table (1000000400)	192.168.1.10:42230	10.0.1.10:80	TCP:A
✕	Feb 27 17:07:07	WAN	virusprot overload table (1000000400)	192.168.1.10:42244	10.0.1.10:80	TCP:A
✕	Feb 27 17:07:07	WAN	virusprot overload table (1000000400)	192.168.1.10:42214	10.0.1.10:80	TCP:PA

Virusprot Table

IP Address
192.168.1.10

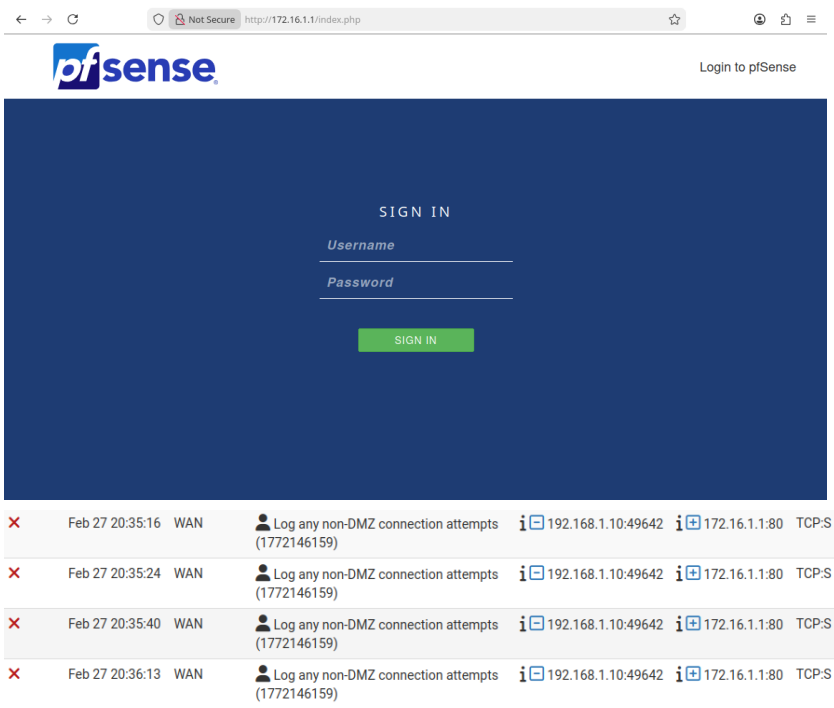
Test 2:
DoS attacks

```
┌──(kali㉿kali)-[~]
└─$ for i in {1..50}; do timeout 30 nc 192.168.1.1 80 & done
[2] 32593
[3] 32594
[4] 32595
```

	
Spoofing attacks: logged	

Series of tests revealed strong security posture. Only an aggressive scan managed to retrieve information of system configuration, but caused the ip address of an attacker to be blocked temporarily by addition to virusprot table. Logging turned out to be effective for any kind of scan except for the stealth one. DoS attack on port 80 failed due to multiple suspicious connection attempts being identified, this led to temporary blockage of attackers IP and successful logging. SYN flood did not result in ip blockage, but was logged. SYN packets are allowed since they are initial for connection initialisation. This can be mitigated by external plugins only.

Administrative Access Tests

Test 1: HTTP to firewall management from LAN should succeed, while from WAN should fail and be logged. (SSH is disabled by default)	

Tests have shown correct administrative access configuration and decent level of logging.

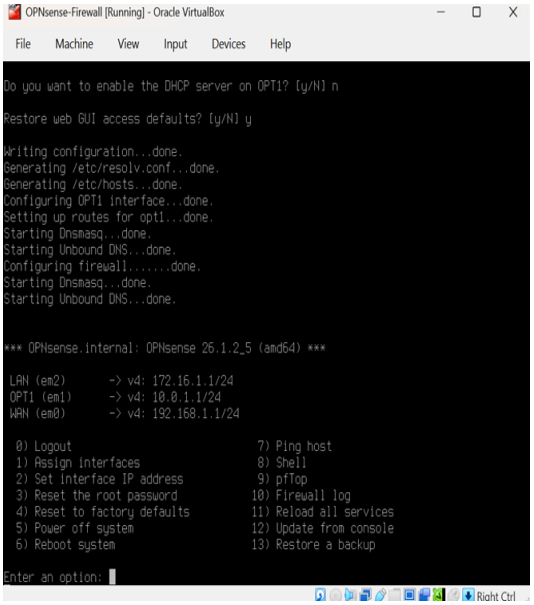
pfSense Strength & Limitations

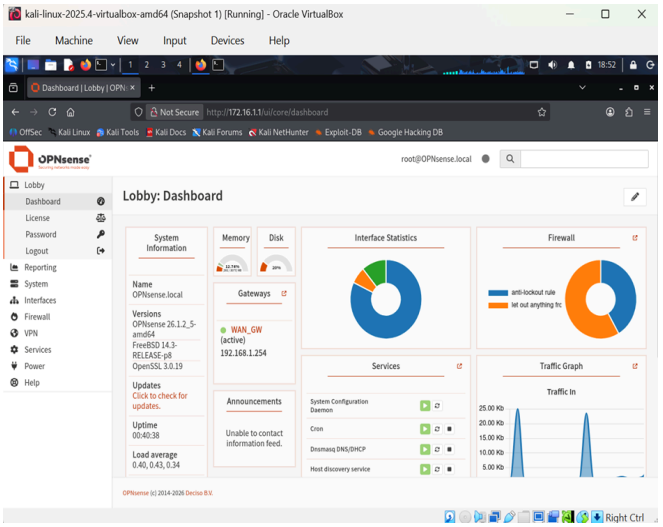
One of the greatest upsides of this firewall is ease of use, it has intuitive management and reasonable default rules. A wide range of features greatly enhances requirements of modern secure networks, one of those is its default ability to identify multiple unsuccessful login and connection attempts for temporary IP blocking. Even though its default configuration is good compared to other products, this does not provide full protection so plugins are still required for production use. This is a stateful firewall, allowing for better tracking of user interactions.

OPNsense

OPNsense is an open source, FreeBSD-based firewall and routing software developed by Deciso. Launched in 2015, it is a fork of pfSense, which in turn was forked from m0n0wall built on FreeBSD.

Implementation & Configuration





OPNsense was set with a default-deny policy. WAN rules allow for inbound HTTP (port 80) to only pass this traffic to the DMZ server through Destination NAT. Administrative SSH from the admin workstation (172.16.1.10) to the DMZ server, HTTP access to the DMZ for all LAN devices and full outbound internet access — with all other LAN-to-DMZ traffic blocked are all enabled with LAN rules. DMZ rules prioritise a block rule preventing any DMZ-to-LAN communication, followed by restricted outbound access for updates (HTTP, HTTPS, DNS, NTP).

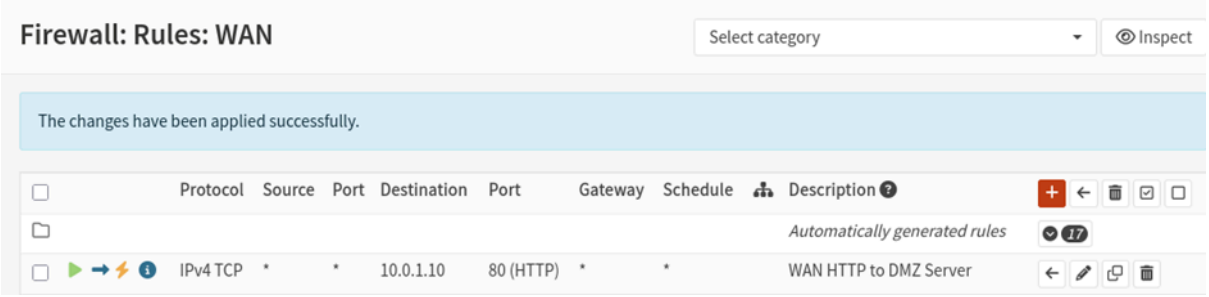
Server Configuration

Apache server started on port 80

```
Feb 25 20:24:23 kali systemd[1]: Starting apache2.service - The Apache HTTP
Feb 25 20:24:23 kali apachectl[3004]: AH00558: apache2: Could not reliably d
Feb 25 20:24:23 kali systemd[1]: Started apache2.service - The Apache HTTP S
```

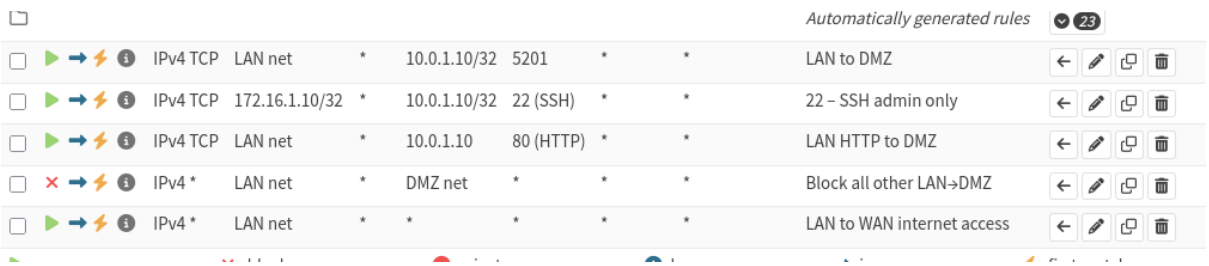
WAN

Rule is allowing incoming HTTP traffic on port 80.



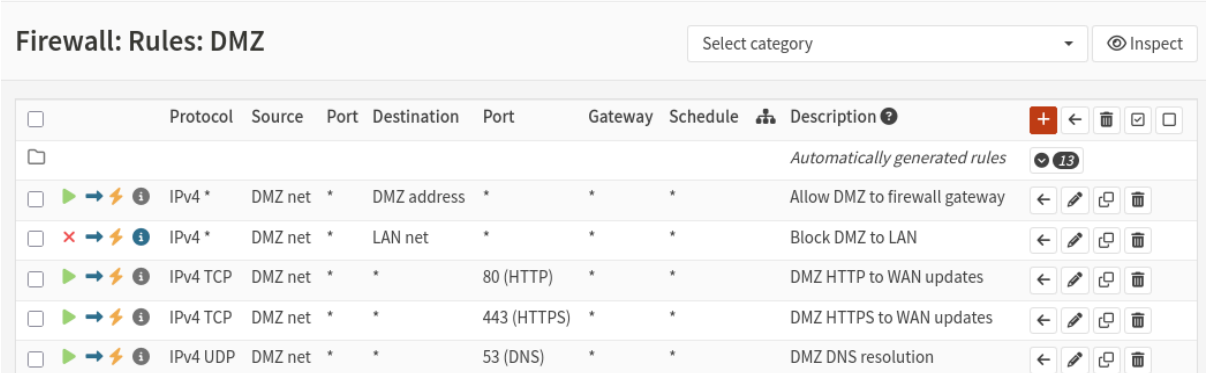
LAN

Admin workstation (172.16.1.10) receives SSH permission for the DMZ server SSH and all LAN devices may remotely reach DMZ web serve. LAN machines can go on the internet. Default “allow all” rules were removed.



DMZ

A block rule at the top precludes any DMZ traffic from reaching the LAN, thus stopping lateral movement if the server is compromised. Subsequent to that, outbound HTTP, HTTPS and DNS are allowed for updates.



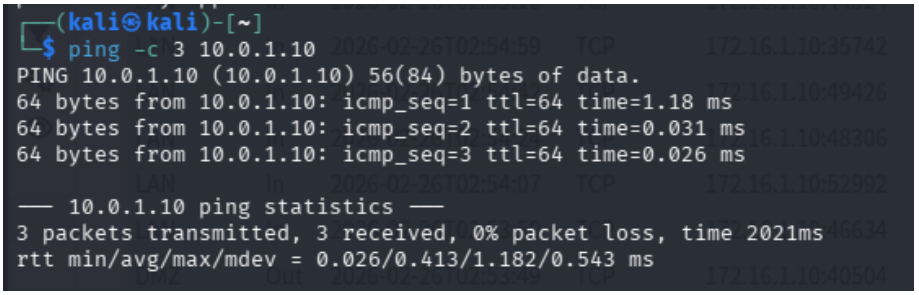
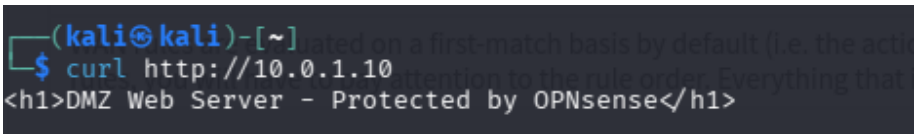
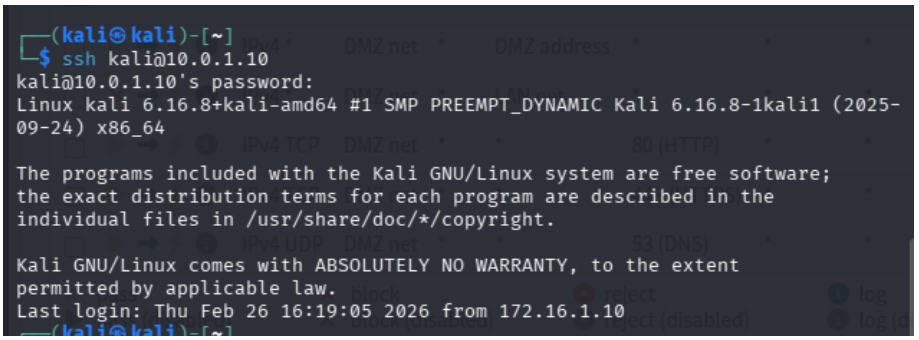
NAT

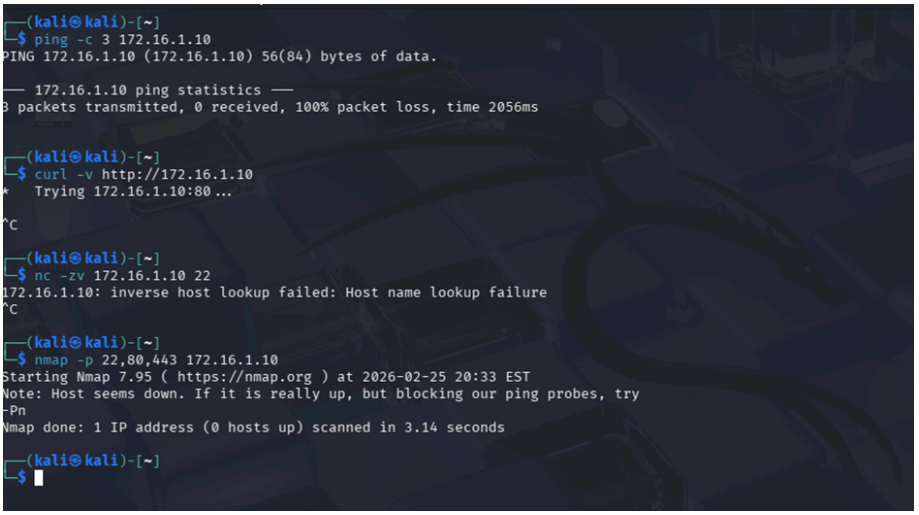
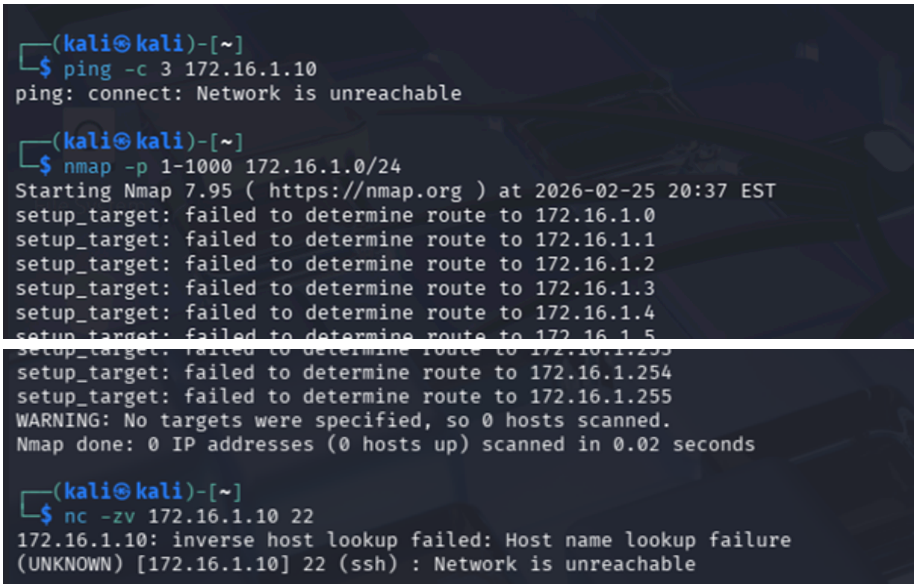
Destination NAT WAN port 80 traffic is forwarded to DMZ server. Outbound NAT uses Hybrid mode with a manual rule to maintain return traffic so it is routed correctly back through the firewall.

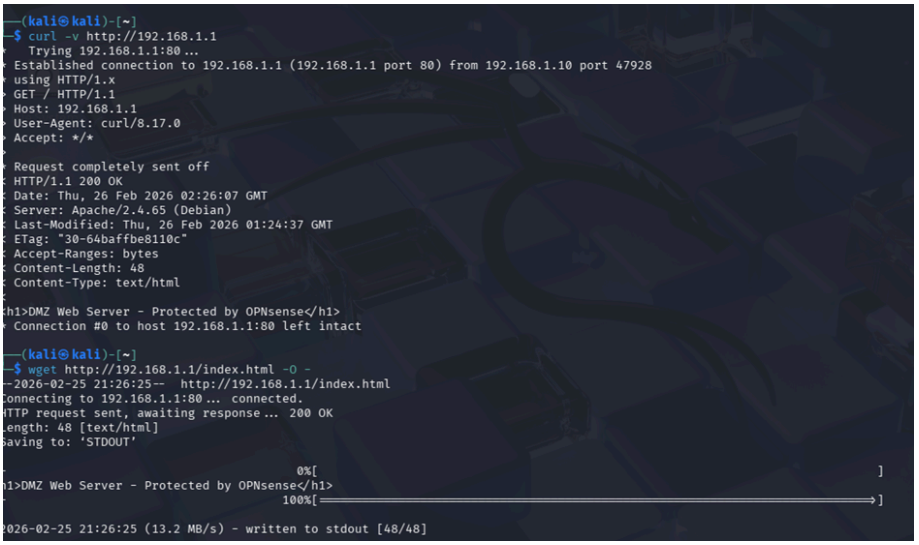
	Version	Protocol	Source	Port	Destination	Port	Redirect Ta...	Redir...	Description	Commands
	LAN	IPv4+IPv6	TCP	*	*	LAN address	80	*	*	Anti-Lockout Rule	
	WAN	*	TCP	*	*	WAN address	http	10.0.1.10	80		

Results & Analysis

Zone Isolation

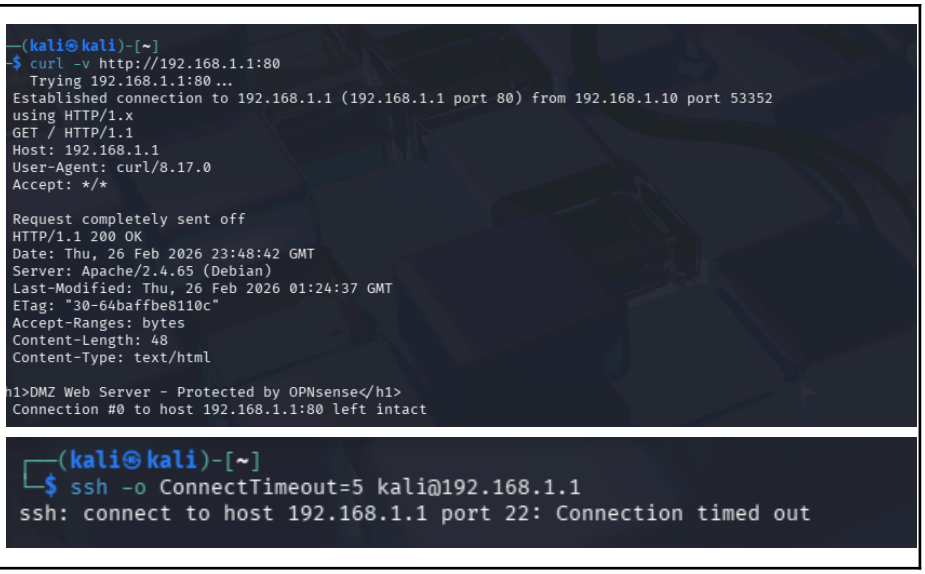
Test 1: LAN → DMZ (should succeed)	  

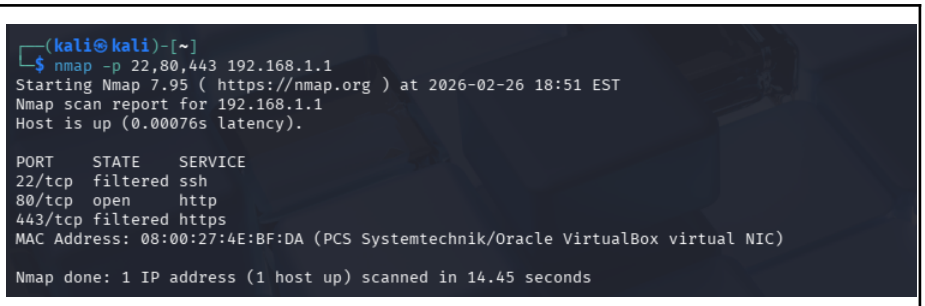
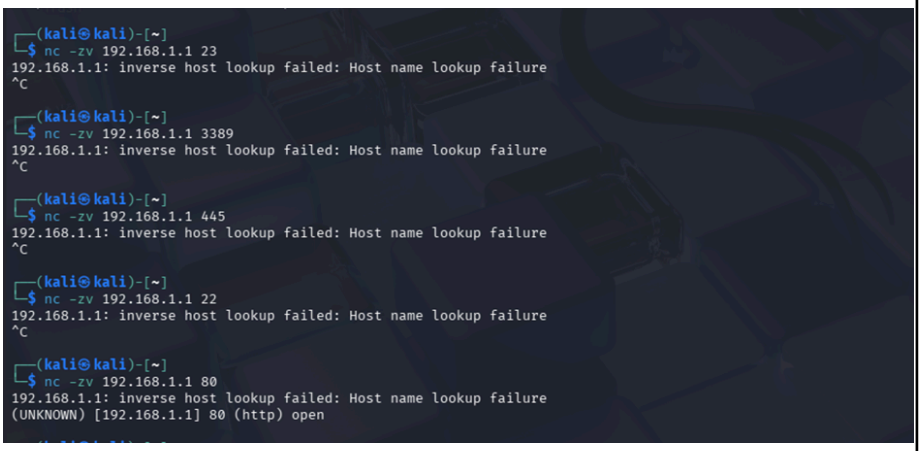
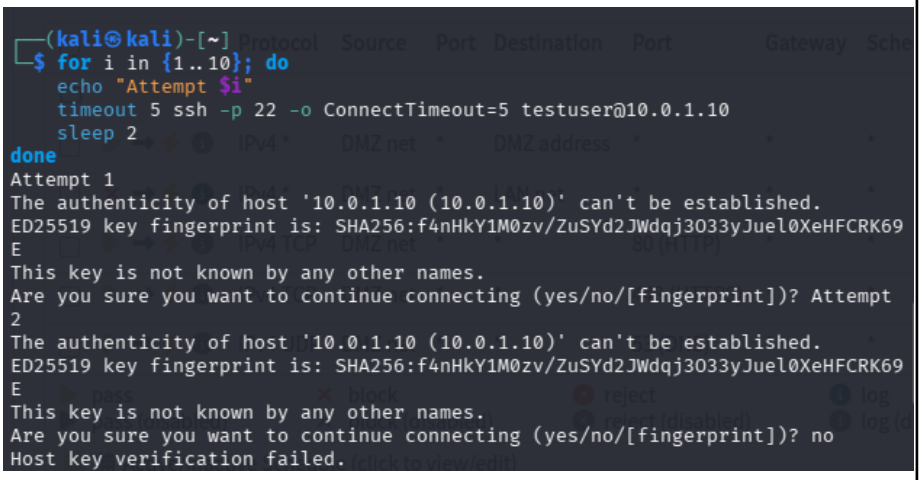
Test 2: DMZ → LAN (should fail)	 
Test 3: WAN → LAN (should fail)	

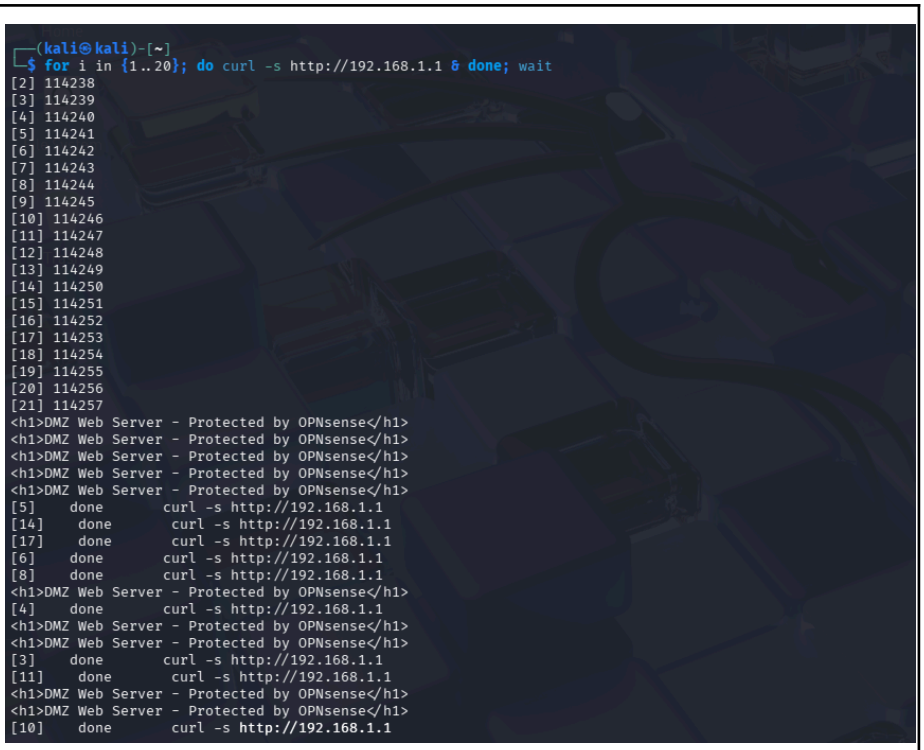
Test 4: WAN → DMZ (should succeed)	

DMZ-LAN isolation was verified through ping, curl and nmap from the DMZ server to the LAN client — all blocked and verified by OPNsense's live log showing "Block DMZ to LAN" entries. WAN-to-LAN was also fully blocked: nmap scan by the attacker revealed no live hosts on the LAN subnet, confirming the internal network was completely invisible externally. Legitimate paths worked correctly: LAN accessed the DMZ for administration, and WAN reached the DMZ web server via NAT.

Service Access Control Testing

Test 1: Authorized Port Access Ran from Attacker-WAN	

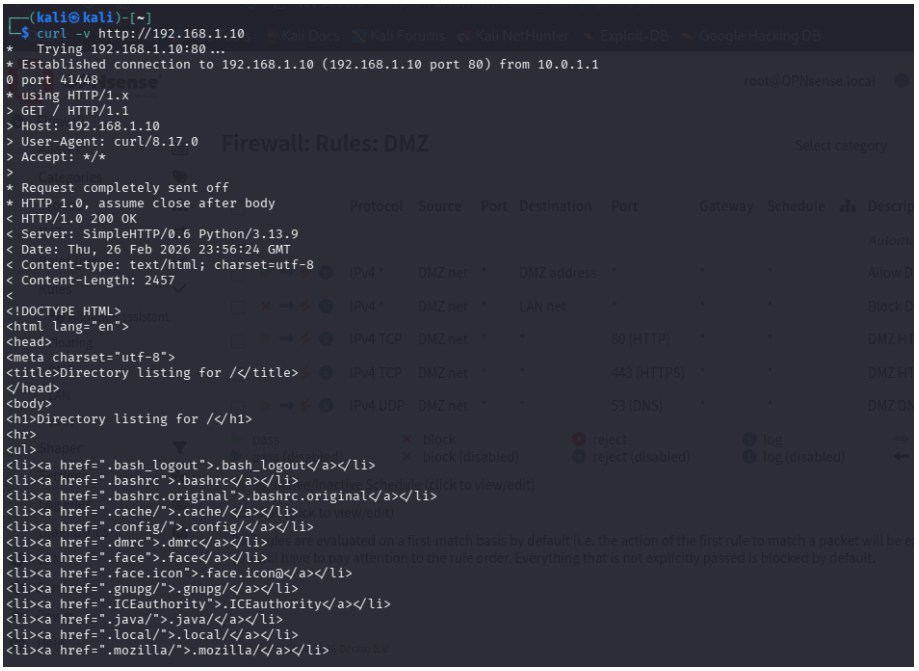
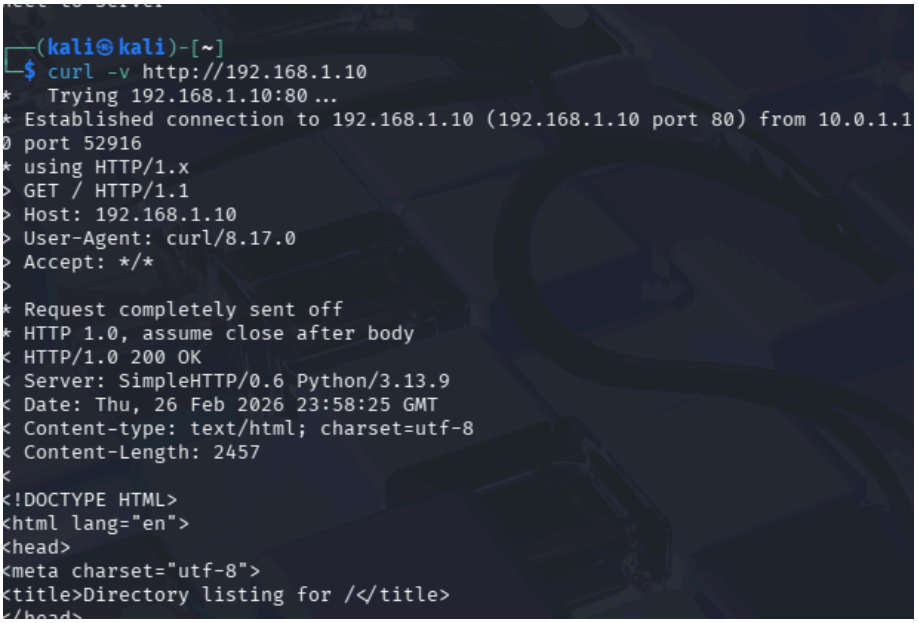
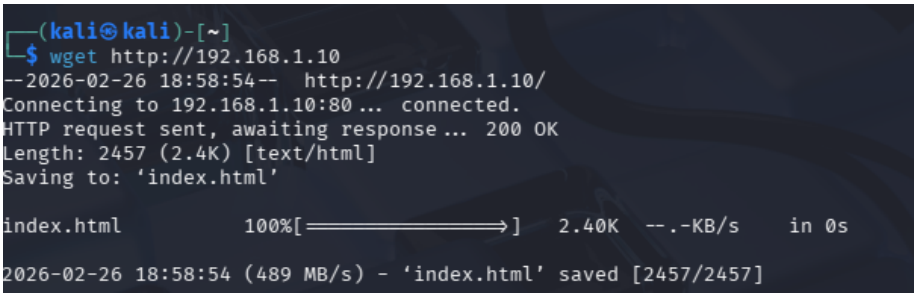
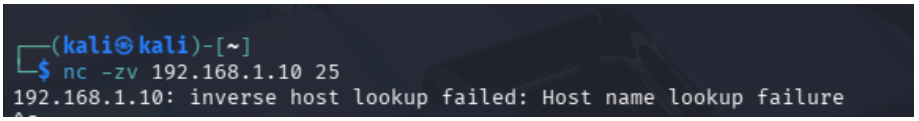
	
Test 2: Blocked Dangerous Ports Ran from Attacker-WAN	
Test 3: Rate Limiting Effectiveness	LAN to DMZ  WAN to DMZ

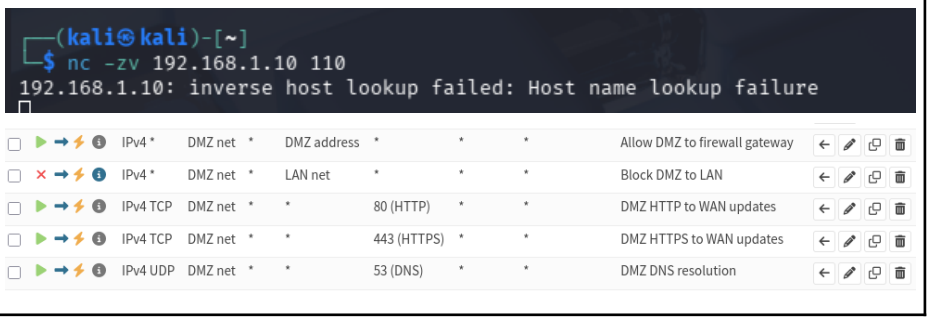
	 HTTP request flooding with 20 concurrent connections was handled without drops, with OPNsense's state table managing concurrent sessions effectively.

Service filtering revealed exact access control. NAT allowed external HTTP access consistently and all unsafe ports, Telnet, SMB, RDP, and SSH were blocked for WAN, returning timeout or connection refused. OPNsense's default-deny ruleset means unexposed ports don't respond.

NAT Functionality Testing

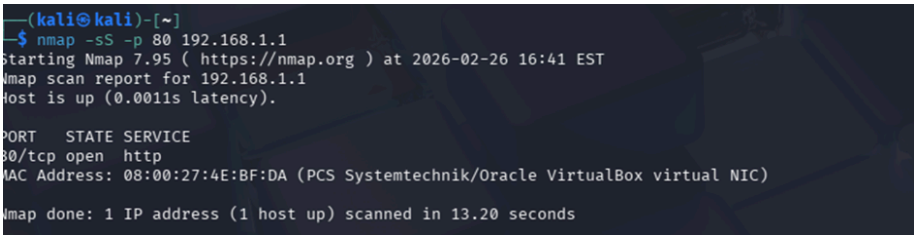
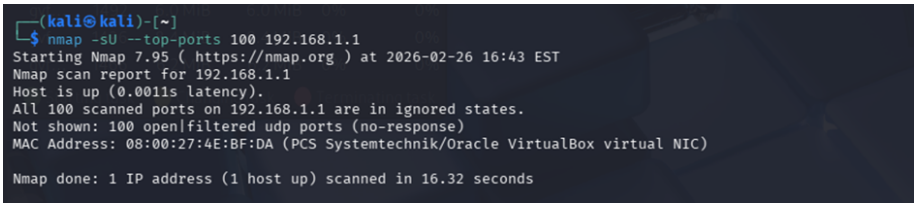
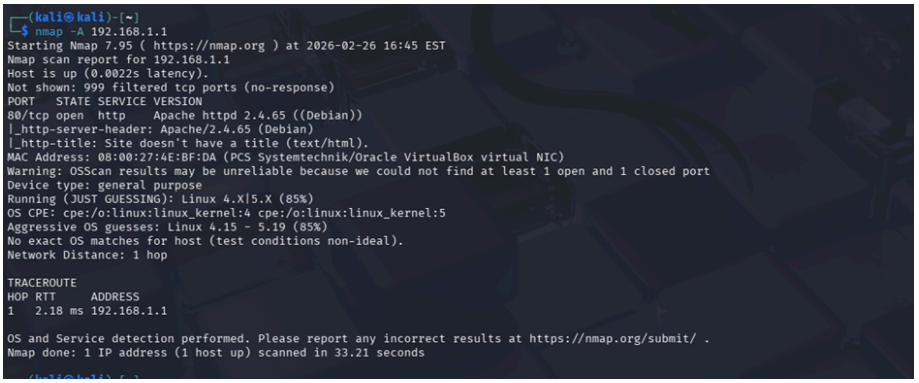
Test 1: LAN Internet Access via NAT Ran from LAN	

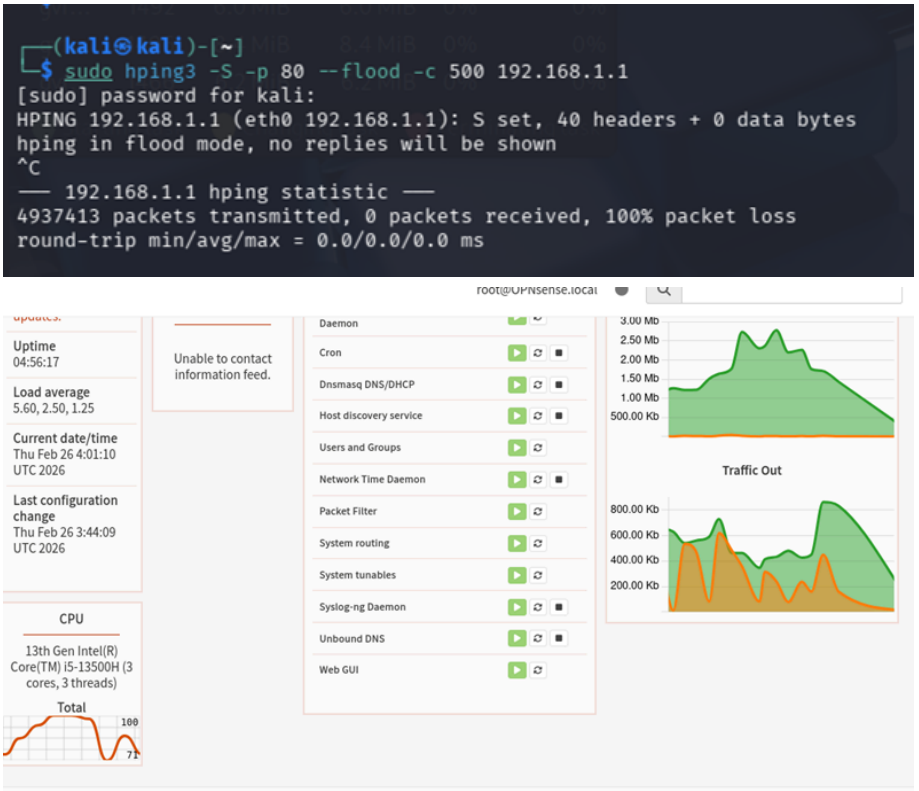
	
Test 2: DMZ Internet Access (Limited) Ran from DMZ	  

NAT functionality worked well as expected. LAN devices accessed the DMZ through the firewall, and the DMZ was only allowed limited outbound access for updates through HTTP, HTTPS, and DNS.

Attack Resistance Testing

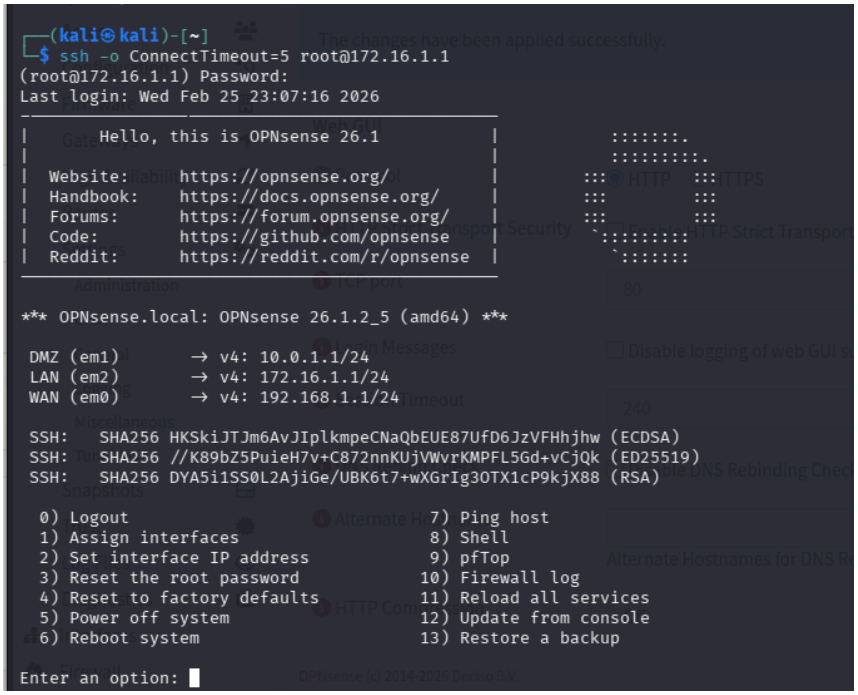
Test 1: Port Scanning Detection Ran from WAN	  

Test 2: Connection Flooding Attack Ran from WAN	

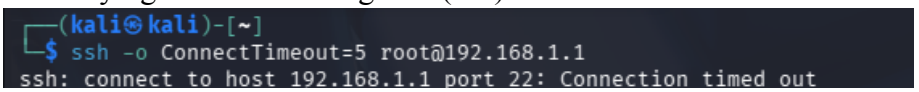
Port scanning from WAN revealed only port 80 is accessible with everything else filtered. So OPNsense hides its identity nicely, Nmap's aggressive scan made it guess that the OS was Linux 4.X instead of FreeBSD. As part of SYN flood testing we threw about 4.9 million packets at it and all were dropped. CPU spiked on the dashboard but all the while LAN still got to reach DMZ server, showing that legitimate traffic remained protected.

Administrative Access Testing

Test 1: SSH Administrative Access	LAN trying to connect using SSH(success)



WAN trying to connect using SSH(fail)



The SSH was successfully allowed to enter the firewall from the LAN admin workstation. No WAN rule allows SSH attempts from the WAN attacker, so remote administrative access can only be accessed from the internal network.

Strengths & Limitations

OPNsense, with its GUI, reduces administrative overhead compared to other nftables-style CLI-based firewalls. Real-time log streaming was useful to ensure rule enforcement, and the alias system implements maintainable policy management. But OPNsense requires more resources than minimal nftables would support, and the GUI can make underlying pf mechanics difficult to understand — NAT return path routing problems in testing would have been more transparent in a CLI environment.

Firewall Comparison

General comparison

Aspect	Nftables	pfSense	OPNsense
Learning curve	Steep	Moderate	Easy
Type	Linux packet filtering framework default in many distributions	Firewall/router OS distribution	Firewall/router OS distribution
Base system	Linux kernel	FreeBSD-based	FreeBSD-based
Management	CLI	GUI/CLI	GUI/CLI
Performance	Extremely high	Good	Good
Community	Community	Enterprise & community (has commercial version)	Community & enterprise
Hardware requirements	Minimal	Moderate	Moderate
Usage	Linux servers, cloud infrastructure, kubernetes	Offices, public wifi, small institutions, high security labs	Offices, public wifi, small institutions, high security labs

Test-based comparison

Since all firewalls are actively used and developed by communities, their performance throughout our tests was fairly similar with minor differences that can be reasoned by their usage purposes. One differing test is response to nmap stealth scan with SYN packets. Reasoning for that is synproxy state set in the rules of pfSense.

Stealth Scan test:

Nftables	pfSense	OPNsense
Does not hide configuration	Hides configuration by temporary source IP block	Does not hide configuration

References

1. Netfilter.org (2024) *nftables wiki*. [Online] Available at: https://wiki.nftables.org/ [Accessed 25 February 2026].
2. Netfilter.org (2023) *Man page of NFT*. [Online] Available at: https://www.netfilter.org/projects/nftables/manpage.html [Accessed 25 February 2026].
3. Netgate (2026) *pfSense Documentation.* Available at: https://docs.netgate.com/pfsense/en/latest/ (Accessed: 25 February 2026).
4. OPNsense (2024) *OPNsense Documentation.* Available at: https://docs.opnsense.org (Accessed: 25 February 2026).
5. Deciso BV (2024) *OPNsense: Open Source Firewall.* Available at: https://docs.opnsense.org/firewall.html (Accessed: 25 February 2026).

Appendix: Comprehensive Test Plan

Detailed Test Scenarios and Commands

Category A: Network Segmentation & Zone Isolation Tests

Test A1: LAN to DMZ Communication

Objective	Expected Result	Test VM
Verify LAN can access DMZ for administration	ALLOW	LAN Client (VM4)

Commands

```
# A1.1 - Ping connectivity
ping -c 3 10.0.1.10

# A1.2 - Web service access
curl -v http://10.0.1.10

# A1.3 - SSH access
ssh user@10.0.1.10

# A1.4 - Traceroute path verification
traceroute 10.0.1.10
```

Test A2: DMZ to LAN Communication (Critical Security Test)

Objective	Expected Result	Test VM
Verify DMZ cannot access LAN (prevent lateral movement)	BLOCK & LOG	DMZ Server (VM3)

Commands	Monitor on Firewall

`# A2.1 - Ping attempt (should fail)`
`ping -c 3 172.16.1.10`
 	
`# A2.2 - Port scan attempt`
	`# Watch for blocked DMZ to LAN traffic`
	`sudo tail -f /var/log/kern.log | grep "DMZ-to-LAN-BLOCKED"`

Commands	Monitor on Firewall
(should fail) nmap -p 22,80,443 172.16.1.10 # A2.3 - Direct connection attempts nc -zv 172.16.1.10 22 nc -zv 172.16.1.10 80 # A2.4 - Traceroute (should stop at firewall) traceroute 172.16.1.10	

Test A3: WAN to LAN Direct Access

Objective	**Expected Result**	**Test VM**
Verify external networks cannot directly access LAN	BLOCK & LOG	Attacker (VM1)

Commands	Monitor on Firewall
# A3.1 - Direct ping to LAN (should fail) ping -c 3 172.16.1.1 ping -c 3 172.16.1.10 # A3.2 - Port scanning LAN nmap -p 1-1000 172.16.1.0/24 # A3.3 - Connection attempts nc -zv 172.16.1.10 22 telnet 172.16.1.10 80	sudo tail -f /var/log/kern.log | grep "FORWARD-DROP"

Test A4: WAN to DMZ Controlled Access

Objective	**Expected Result**	**Test VM**
Verify external access to authorized DMZ services	ALLOW (HTTP/HTTPS only)	Attacker (VM1)

Commands

```
# A4.1 - Web service access (should work)
curl -v http://192.168.1.1
wget http://192.168.1.1/index.html

# A4.2 - HTTPS access (if configured)
curl -k https://192.168.1.1

# A4.3 - Unauthorized ports (should fail)
nc -zv 192.168.1.1 22
nc -zv 192.168.1.1 23
nc -zv 192.168.1.1 3389
telnet 192.168.1.1 25
```

Category B: Service Access Control Tests

Test B1: Authorized Port Access

Objective	Expected Result	Test VM
Verify only intended services are accessible	ALLOW	Attacker (VM1)

Commands

```
# B1.1 - HTTP access (port 80) - should work
curl -v http://192.168.1.1:80
curl -v http://10.0.1.10:80

# B1.2 - SSH access (port 22) - should fail
ssh user@192.168.1.1
ssh user@10.0.1.10

# B1.3 - Service discovery
nmap -p 22,80,443 192.168.1.1
```

Test B2: Blocked Dangerous Ports

Objective	Expected Result	Test VM
Verify dangerous ports are blocked	BLOCK & LOG	Attacker (VM1)

Commands	Monitor on Firewall

`# B2.1 - Telnet (port 23)`
`nc -zv 192.168.1.1 23`
 	
`# B2.2 - NetBIOS (port 139)`
`nc -zv 192.168.1.1 139`
 	
`# B2.3 - SMB (port 445)`
`nc -zv 192.168.1.1 445`
 	
`# B2.4 - RDP (port 3389)`
`nc -zv 192.168.1.1 3389`
 	
`# B2.5 - SQL Server (port 1433)`
`nc -zv 192.168.1.1 1433`
	`sudo tail -f /var/log/kern.log | grep "INPUT-DROP"`

Test B3: Rate Limiting Effectiveness

Objective	**Test VM**
Verify rate limiting protects against brute force	LAN Client (VM4) & Attacker (VM1)

Commands

```
# B3.1 - SSH rate limiting test (from LAN → DMZ)
for i in {1..10}; do
    echo "Attempt $i"
    timeout 5 ssh -p 22 -o ConnectTimeout=5 testuser@10.0.1.10
    sleep 2
done

# B3.2 - HTTP request flooding (from WAN → DMZ)
for i in {1..20}; do
    curl -s http://192.168.1.1 &
done
wait
```

Category C: NAT Functionality Tests

Test C1: LAN Internet Access via NAT

Objective	**Test VM**
Verify LAN can access external services through NAT	LAN Client (VM4)

Commands	On Attacker VM (VM1) - Prepare test server

`# C1.1 - Basic connectivity to "internet" (attacker VM)`
`ping -c 3 192.168.1.10`
 	
`# C1.2 - HTTP access to external service`
`curl -v http://192.168.1.10`
	`# Start simple HTTP server`
	`python3 -m http.server 80`
	
	`# Or netcat listener to see source IPs`
	`nc -l -p 8080`

Test C2: DMZ Internet Access (Limited)

Objective	**Test VM**
Verify DMZ has limited outbound access	DMZ Server (VM3)

Commands

`# C2.1 - Allowed outbound (HTTP/HTTPS)`
`curl -v http://192.168.1.10`
`wget http://192.168.1.10`
 	
`# C2.2 - Blocked outbound ports`
`nc -zv 192.168.1.10 25`
`nc -zv 192.168.1.10 110`

Category D: Attack Resistance Tests

Test D1: Port Scanning Detection

Objective	Test VM
Test firewall response to reconnaissance	Attacker (VM1)

Commands

```
# D1.1 - TCP SYN scan
nmap -sS -p 1-1000 192.168.1.1

# D1.2 - UDP scan
nmap -sU --top-ports 100 192.168.1.1

# D1.3 - TCP scan
nmap -sN -p 1-65535 192.168.1.1

# D1.4 - OS detection
nmap -O 192.168.1.1

# D1.5 - Aggressive scan
nmap -A 192.168.1.1
```

Test D2: Connection Flooding Attack

Objective	Test VM
Test resistance to DoS attacks	Attacker (VM1)

Commands

```
# D2.1 - TCP connection flood
for i in {1..50}; do
    timeout 30 nc 192.168.1.1 80 &
done

# D2.2 - SYN flood (requires hping3)
sudo hping3 -S -p 80 --flood -c 1000 192.168.1.1

# D2.3 - UDP flood
```

Commands
`sudo hping3 --udp -p 53 --flood -c 1000 192.168.1.1`

Test D3: Anti-Spoofing Protection

Objective	**Test VM**
Verify spoofed IP protection	Attacker (VM1)

Commands	Monitor on Firewall

`# D3.1 - Spoof private IP addresses`
`sudo hping3 -S -a 10.0.1.5 -p 80 192.168.1.1`
`sudo hping3 -S -a 172.16.1.5 -p 80 192.168.1.1`
`sudo hping3 -S -a 127.0.0.1 -p 80 192.168.1.1`
	`sudo tail -f /var/log/kern.log | grep "SPOOFED-SRC"`

Category E: Administrative Access Tests

Test E1: SSH Administrative Access

Objective	**Test VM**
Verify secure administrative access (SSH to firewall)	LAN Client (VM4) & Attacker (VM1)

From LAN (Should Work)	From WAN (Should Fail)

`# E1.1 - SSH from LAN`
`ssh admin@172.16.1.1`
 	
`# E1.2 - Rate limiting test from LAN`
`for i in {1..8}; do`
`  ssh -o ConnectTimeout=5 admin@172.16.1.1;`
`done`
	`# E1.3 - SSH from WAN (should be blocked)`
	`ssh admin@192.168.1.1`

Monitoring Commands During Testing

Real-time Log Monitoring

```
# General firewall logs
sudo tail -f /var/log/kern.log

# Specific rule logging
sudo tail -f /var/log/kern.log | grep -E
"(INPUT-DROP|FORWARD-DROP|DMZ-to-LAN-BLOCKED|BLOCKED-PORT|SPOOFED-
SRC)"
```

Traffic Capture

```
# Capture traffic on specific interfaces
sudo tcpdump -i enp0s3 -w wan_traffic.pcap &
sudo tcpdump -i enp0s8 -w dmz_traffic.pcap &
sudo tcpdump -i enp0s9 -w lan_traffic.pcap &

# Stop captures
sudo killall tcpdump
```

Appendix: Commands and Configurations (nftables)

Initial System Setup

Nftables Installation and Verification

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install nftables (usually pre-installed on Ubuntu 22.04)
sudo apt install nftables

# Verify installation
sudo nft --version
nftables v1.0.9 (Old Doc Yak #3)

# Check initial ruleset status
sudo nft list ruleset

# Enable nftables service
sudo systemctl enable nftables
sudo systemctl start nftables
```

Network Interface Configuration

Firewall VM Network Configuration (/etc/netplan/01-network.yaml)

```
network:
  version: 2
  ethernets:
    enp0s3:  # WAN interface
      dhcp4: false
      addresses: [192.168.1.1/24]
      routes:
        - to: 0.0.0.0/0
          via: 192.168.1.254
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
    enp0s8:  # DMZ interface
      dhcp4: false
      addresses: [10.0.1.1/24]
    enp0s9:  # LAN interface
      dhcp4: false
      addresses: [172.16.1.1/24]
```

Apply network configuration

```
sudo netplan apply
sudo ip addr show  # Verify interfaces
```

Enable IP Forwarding

```
# Enable IP forwarding temporarily
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1

# Make permanent
echo 'net.ipv4.ip_forward = 1' | sudo tee -a /etc/sysctl.conf
echo 'net.ipv6.conf.all.forwarding = 1' | sudo tee -a
/etc/sysctl.conf

# Apply changes
sudo sysctl -p
```

Complete Nftables Configuration

Main Firewall Rules (/etc/nftables.conf)

```
#!/usr/sbin/nft -f

# Clear any existing rules
flush ruleset

# Define main filter table
table inet filter {
    # Define network sets for organization
    set wan_networks {
        type ipv4_addr
        flags interval
        elements = { 192.168.1.0/24 }
    }

    set dmz_networks {
        type ipv4_addr
        flags interval
        elements = { 10.0.1.0/24 }
    }

    set lan_networks {
        type ipv4_addr
        flags interval
        elements = { 172.16.1.0/24 }
    }

    set web_ports {
        type inet_service
        elements = { 80, 443 }
    }

    set blocked_ports {
        type inet_service
        elements = { 23, 135, 139, 445, 1433, 3389 }
    }

    # Input chain - traffic destined for the firewall
    chain input {
```

```
        type filter hook input priority filter; policy drop;

        # Allow loopback traffic
        iif lo accept comment "Allow loopback"

        # Allow established and related connections
        ct state established,related accept comment "Allow
established connections"

        # Block WAN direct access to firewall interfaces
        iif "enp0s3" ip daddr { 192.168.1.1, 10.0.1.1, 172.16.1.1
} counter log prefix "WAN-TO-FW-BLOCKED: " drop comment "Block WAN
to firewall direct access"

        # Block ping from WAN to firewall
        iif "enp0s3" icmp type echo-request counter log prefix
"WAN-PING-BLOCKED: " drop comment "Block WAN ping to firewall"

        # Allow ICMP for network diagnostics
        icmp type { echo-request, destination-unreachable,
time-exceeded } limit rate 10/second accept
        icmpv6 type { echo-request, destination-unreachable,
time-exceeded, nd-neighbor-solicit, nd-neighbor-advert } accept

        # Allow SSH from LAN to firewall
        iif "enp0s9" ip daddr 172.16.1.1 tcp dport 22 ct state new
limit rate 5/minute accept comment "SSH from LAN with rate
limiting"

        # Block SSH from DMZ to firewall
        iif "enp0s8" tcp dport 22 counter log prefix
"DMZ-SSH-BLOCKED: " drop comment "Block DMZ SSH to firewall"

        # Block SSH from WAN to firewall
        iif "enp0s3" tcp dport 22 counter log prefix
"WAN-SSH-BLOCKED: " drop comment "Block WAN SSH to firewall"

        # Allow DHCP if needed
        udp sport 68 udp dport 67 accept comment "DHCP client"

        # Allow NTP for time synchronization
        udp dport 123 accept comment "NTP"
```

```
        # Log and count dropped packets
        counter log prefix "INPUT-DROP: " drop
    }

    # Forward chain - traffic passing through the firewall
    chain forward {
        type filter hook forward priority filter; policy drop;

        # Allow established and related connections
        ct state established,related accept comment "Allow
established connections"

        # WAN to DMZ - Allow web traffic to web server
        iif "enp0s3" oif "enp0s8" ip daddr 10.0.1.10 tcp dport
@web_ports ct state new limit rate 20/second burst 40 packets
accept comment "Web traffic to DMZ (with rate limitting)"

        # LAN to DMZ - Allow SSH with rate limiting (for
administration)
        iif "enp0s9" oif "enp0s8" ip daddr 10.0.1.10 tcp dport 22
ct state new limit rate 3/minute accept comment "SSH to DMZ with
rate limiting"

        # LAN to DMZ - Allow all traffic (internal users)
        iif "enp0s9" oif "enp0s8" tcp dport != 22 accept comment
"LAN to DMZ access (no SSH)"

        # LAN to WAN - Allow internet access
        iif "enp0s9" oif "enp0s3" accept comment "LAN to internet"

        # DMZ to WAN - Limited outbound access for updates
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 tcp dport {
80, 443, 53 } accept comment "DMZ outbound web/DNS"
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 udp dport
53 accept comment "DMZ outbound DNS"
        iif "enp0s8" oif "enp0s3" ip saddr 10.0.1.0/24 udp dport
123 accept comment "DMZ NTP"

        # Block DMZ to LAN (critical security rule)
        iif "enp0s8" oif "enp0s9" counter log prefix
"DMZ-to-LAN-BLOCKED: " drop comment "Block DMZ to LAN"
```

```
        # Block dangerous ports from WAN
        iif "enp0s3" tcp dport @blocked_ports counter log prefix
"BLOCKED-PORT: " drop comment "Block dangerous ports"

        # Anti-spoofing rules
        iif "enp0s3" ip saddr { 10.0.0.0/8, 172.16.0.0/12,
192.168.0.0/16, 127.0.0.0/8 } counter log prefix "SPOOFED-SRC: "
drop comment "Block spoofed private IPs from WAN"

        # Log and count all other drops
        counter log prefix "FORWARD-DROP: " drop
    }

    # Output chain - traffic originating from the firewall
    chain output {
        type filter hook output priority filter; policy accept;

        # Log outbound connections for monitoring
        # tcp dport { 80, 443 } counter log prefix "FIREWALL-OUT:
"
    }
}

# NAT table for address translation
table ip nat {
    # Define web ports set in NAT table
    set web_ports {
        type inet_service
        elements = { 80, 443 }
    }

    # DNAT chain for incoming connections
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;

        # Port forward web traffic to DMZ server
        iif "enp0s3" tcp dport @web_ports counter dnat to
10.0.1.10 comment "Forward web to DMZ"
    }

    # SNAT chain for outgoing connections
```

```
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;

        # Masquerade LAN and DMZ traffic to WAN
        oif "enp0s3" ip saddr { 172.16.1.0/24, 10.0.1.0/24 }
counter masquerade comment "NAT internal networks"
    }
}

# Logging table for detailed monitoring
table inet logging {
    set suspicious_ips {
        type ipv4_addr
        size 1000
        flags timeout
        timeout 1h
    }

    chain log_suspicious {
        # Add IPs that exceed connection limits
        add @suspicious_ips { ip saddr limit rate over 100/minute
}
        ip saddr @suspicious_ips counter log prefix
"SUSPICIOUS-IP: " drop
    }
}
```

Apply and Verify Configuration

```
# Test configuration syntax
sudo nft -c -f /etc/nftables.conf

# Apply configuration
sudo nft -f /etc/nftables.conf

# Verify rules are loaded
sudo nft list ruleset

# Make configuration persistent
sudo systemctl reload nftables

# Check service status
sudo systemctl status nftables
```

DMZ Server Configuration

Web Server Setup (VM3 - DMZ)

```
# Update and install Apache
sudo apt update
sudo apt install apache2 -y

# Configure network (if not using DHCP)
sudo ip addr add 10.0.1.10/24 dev enp0s3
sudo ip route add default via 10.0.1.1

# Make network config persistent
cat << EOF | sudo tee /etc/netplan/01-network.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses: [10.0.1.10/24]
      routes:
        - to: 0.0.0.0/0
          via: 10.0.1.1
```

```
      nameservers:
        addresses: [8.8.8.8]
EOF

sudo netplan apply

# Create test web content
cat << EOF | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
    <title>DMZ Web Server</title>
</head>
<body>
    <h1>DMZ Web Server Test Page</h1>
    <p>Server: $(hostname)</p>
    <p>IP Address: $(ip addr show enp0s3 | grep 'inet ' | awk
'{print $2}')</p>
    <p>Timestamp: $(date)</p>
    <hr>
    <p>This server is protected by nftables firewall</p>
</body>
</html>
EOF

# Create a test PHP page (optional)
sudo apt install php libapache2-mod-php -y
cat << EOF | sudo tee /var/www/html/info.php
<?php
phpinfo();
?>
EOF

# Enable and start Apache
sudo systemctl enable apache2
sudo systemctl start apache2

# Verify service
sudo systemctl status apache2
curl localhost
```

SSH Server Hardening

```
# Backup original SSH config
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

# Configure SSH security
cat << EOF | sudo tee -a /etc/ssh/sshd_config

# Security hardening
Port 22
PermitRootLogin no
PasswordAuthentication yes
PubkeyAuthentication yes
MaxAuthTries 3
MaxSessions 2
ClientAliveInterval 300
ClientAliveCountMax 2
LoginGraceTime 60
Protocol 2
EOF

# Restart SSH service
sudo systemctl restart sshd
```

LAN Client Configuration

Network Setup (VM4 - LAN)

```
# Configure static IP
cat << EOF | sudo tee /etc/netplan/01-network.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses: [172.16.1.10/24]
      routes:
        - to: 0.0.0.0/0
          via: 172.16.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
EOF

sudo netplan apply
```

WAN Client Configuration

Set static IP (VM4 - WAN/Attacker)

```
sudo ip addr add 192.168.1.10/24 dev eth0
sudo ip route add default via 192.168.1.1
```